

Лекция. Применение темы Material Design с помощью Jetpack Compose

Material Design - это система дизайна, созданная и поддерживаемая дизайнерами и разработчиками Google для создания высококачественного цифрового опыта для Android, а также других мобильных и веб-платформ. Она содержит рекомендации по созданию читаемого, привлекательного и последовательного пользовательского интерфейса приложений.

В этом уроке вы узнаете о Material Theming, который позволяет использовать Material Design в вашем приложении, с руководством по настройке цветов, типографики и форм. Вы можете настроить для своего приложения как можно меньше или больше. Вы также узнаете, как добавить верхнюю панель приложения для отображения его названия и значка.

Необходимые условия

- Знакомство с языком Kotlin, включая синтаксис, функции и переменные.
- Уметь создавать макеты в Compose, включая строки и столбцы с подложкой.
- Уметь создавать простые списки в Compose.

Что вы узнаете

- Как применить Material Theming к приложению Compose.
- Как добавить в приложение пользовательскую цветовую палитру.
- Как добавлять пользовательские шрифты в приложение.
- Как добавлять пользовательские формы к элементам приложения.
- Как добавить в приложение верхнюю панель приложений.

Что вы создадите

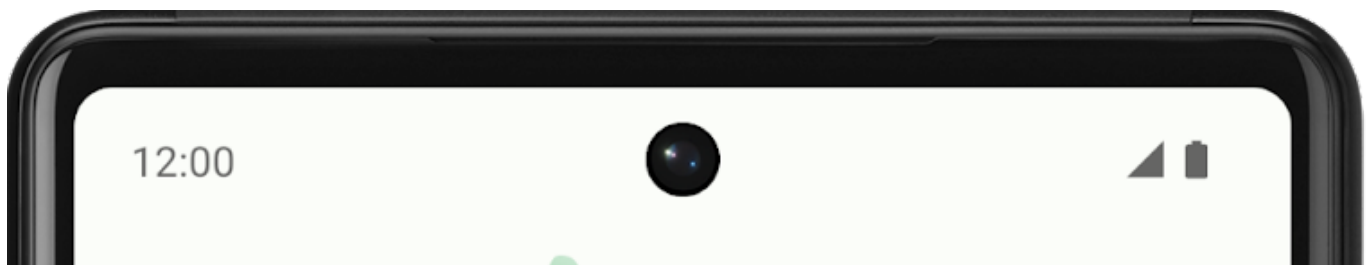
- Вы создадите красивое приложение, в котором будут использованы лучшие практики Material Design.

Что вам понадобится

- Последняя версия Android Studio.
- Подключение к gogs для загрузки начального кода и шрифтов.

Обзор приложения

В этом уроке вы создадите **Woof**, приложение, которое отображает список собак и использует Material Design для создания красивого интерфейса приложения.





Koda

2 years old



Lola

16 years old



Frankie

2 years old



Nox

8 years old



Faye

8 years old



Bella

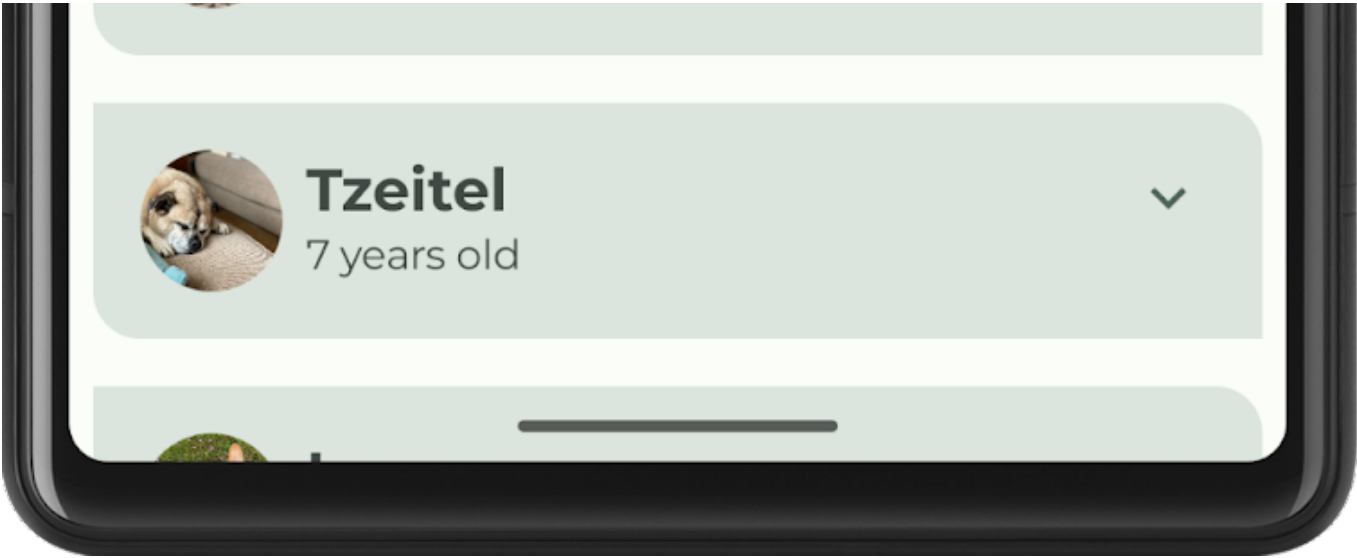
14 years old



Moana

2 years old





В этом уроке мы покажем вам некоторые возможности использования Material Theming. Используйте этот коделаб для поиска идей о том, как использовать Material Theming для улучшения внешнего вида и ощущения приложений, которые вы создаете в будущем.

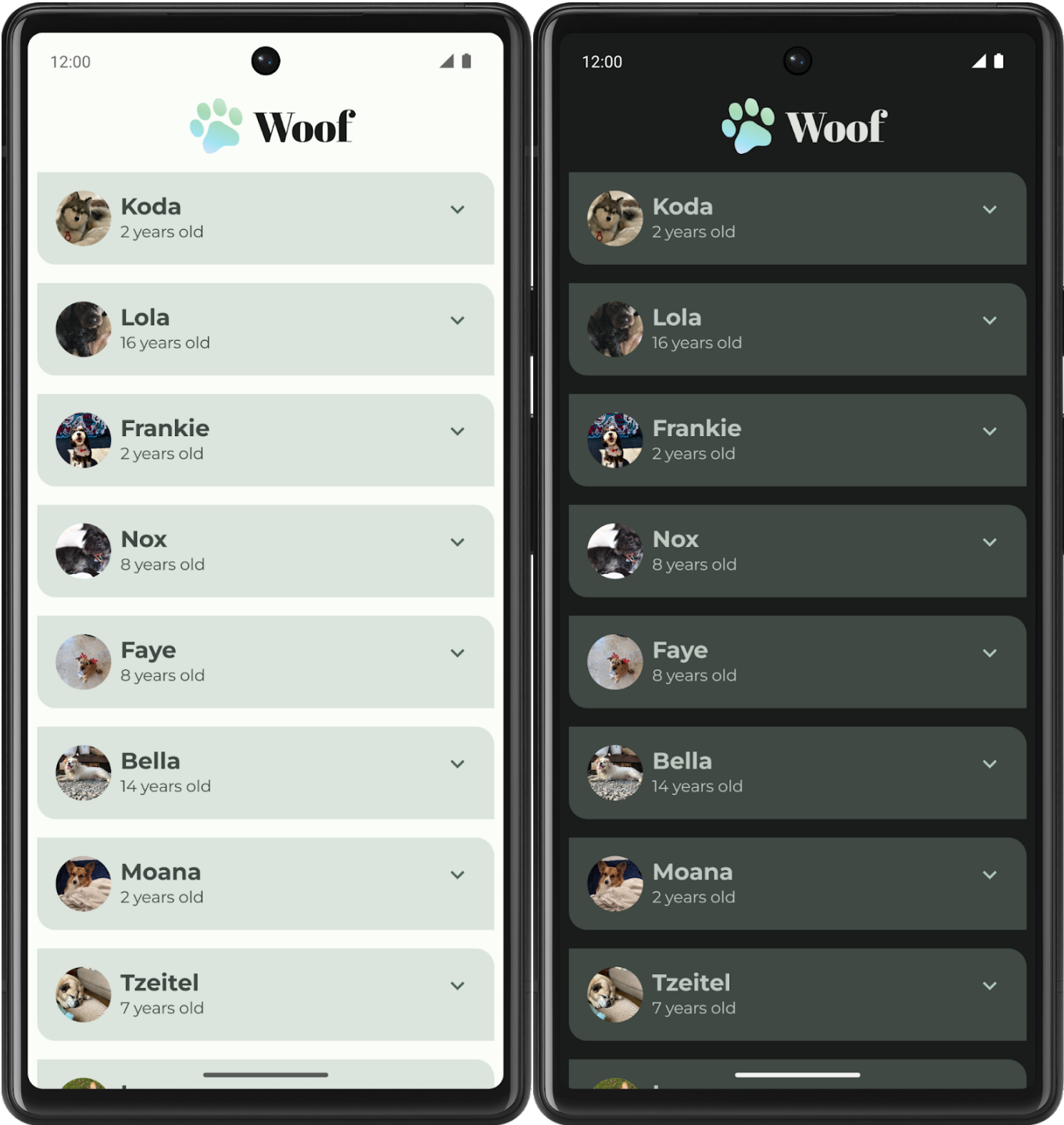
Цветовая палитра

Ниже приведены цветовые палитры для светлой и темной тем, которые мы создадим.

Light Scheme			
Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Dark Scheme			
Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Вот финальная версия приложения в светлой и темной темах.



Типографика

Ниже приведены стили шрифтов, которые вы будете использовать в приложении.

Type Role	Font	Font weight	Size
displayLarge	Abril Fatface	Normal	36sp
displayMedium	Montserrat	Bold	20sp
labelSmall	Montserrat	Bold	14sp
bodyLarge	Montserrat	Normal	14sp

Файл темы

Файл `Theme.kt` - это файл, содержащий всю информацию о теме приложения, которая определяется с помощью цвета, типографики и формы. Это важный файл, который вы должны знать. Внутри файла находится составная функция `WoofTheme()`, которая задает цвета, типографику и формы приложения.

```

@Composable
fun WoofTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
        }

        darkTheme -> DarkColors
        else -> LightColors
    }
    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            setUpEdgeToEdge(view, darkTheme)
        }
    }

    MaterialTheme(
        colorScheme = colorScheme,
        shapes = Shapes,
        typography = Typography,

```

```

        content = content
    )
}

/**
 * Устанавливает от края до края для окна этого [вида]. Цвета системных значков
 * устанавливаются либо
 * светлыми или темными в зависимости от того, включена ли тема [darkTheme] или
 * нет.
 */
private fun setUpEdgeToEdge(view: View, darkTheme: Boolean) {
    val window = (view.context as Activity).window
    WindowCompat.setDecorFitsSystemWindows(window, false)
    window.statusBarColor = Color.Transparent.toArgb()
    val navigationBarColor = when {
        Build.VERSION.SDK_INT >= 29 -> Color.Transparent.toArgb()
        Build.VERSION.SDK_INT >= 26 -> Color(0xFF, 0xFF, 0xFF, 0x63).toArgb()
        // Min sdk version for this app is 24, this block is for SDK versions 24
and 25
        else -> Color(0x00, 0x00, 0x00, 0x50).toArgb()
    }
    window.navigationBarColor = navigationBarColor
    val controller = WindowCompat.getInsetsController(window, view)
    controller.isAppearanceLightStatusBars = !darkTheme
    controller.isAppearanceLightNavigationBars = !darkTheme
}

```

В файле MainActivity.kt добавляется функция WoofTheme(), которая обеспечивает материальную тематику для всего приложения.

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            WoofTheme {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ) {
                    WoofApp()
                }
            }
        }
    }
}

```

- Взгляните на WoofPreview(). Функция WoofTheme() добавлена, чтобы обеспечить оформление материала, которое вы видите в WoofPreview().

```
@Preview
@Composable
fun WoofPreview() {
    WoofTheme(darkTheme = false) {
        WoofApp()
    }
}
```

Получите стартовый код

Чтобы начать работу, загрузите стартовый код:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-
compose-training-woof.git
$ cd basic-android-kotlin-compose-training-woof
$ git checkout starter
```

Настройка: для работы измените параметры в проекте

AGP: 7.4.1 Gradle version: 7.5

build.gradle.kts (project)

```
plugins { id("com.android.application") version "7.4.1" apply false id("com.android.library") version "7.4.1"
apply false id("org.jetbrains.kotlin.android") version "2.0.0" apply false id("org.jetbrains.kotlin.plugin.compose")
version "2.0.0" apply false }
```

- версия core зависимости

```
implementation("androidx.core:core-ktx:1.12.0")
```

Изучите стартовый код

- Откройте стартовый код в Android Studio.
- Откройте `com.example.woof > data > Dog.kt`. Здесь содержится класс данных `Dog`, который будет использоваться для представления фотографии, имени, возраста и увлечений собаки. Здесь также содержится список собак и информация, которую вы будете использовать в качестве данных в своем приложении.
- Откройте `res > drawable`. Здесь содержатся все изображения, необходимые для этого проекта, включая значок приложения, изображения собак и иконки.
- Откройте `res > values > strings.xml`. Здесь содержатся строки, которые вы используете в этом приложении, включая название приложения, клички собак, их описания и многое другое.

- Откройте файл MainActivity.kt. В нем содержится код для создания простого списка, в котором отображается фотография собаки, ее имя и возраст.

WoofApp() содержит LazyColumn, который отображает DogItems. DogItem() содержит строку, в которой отображается фотография собаки и информация о ней. DogIcon() отображает фотографию собаки. DogInformation() отображает кличку и возраст собаки. Функция WoofPreview() позволяет увидеть предварительный просмотр приложения в панели Design.

Примечание: Вы можете заметить, что в каждом методе @Composable в качестве параметра добавлен модификатор. В Compose наилучшей практикой является передача параметра модификатора в композитные функции. Это позволяет родительской композитной функции передавать контекстную информацию дочерней композитной функции.

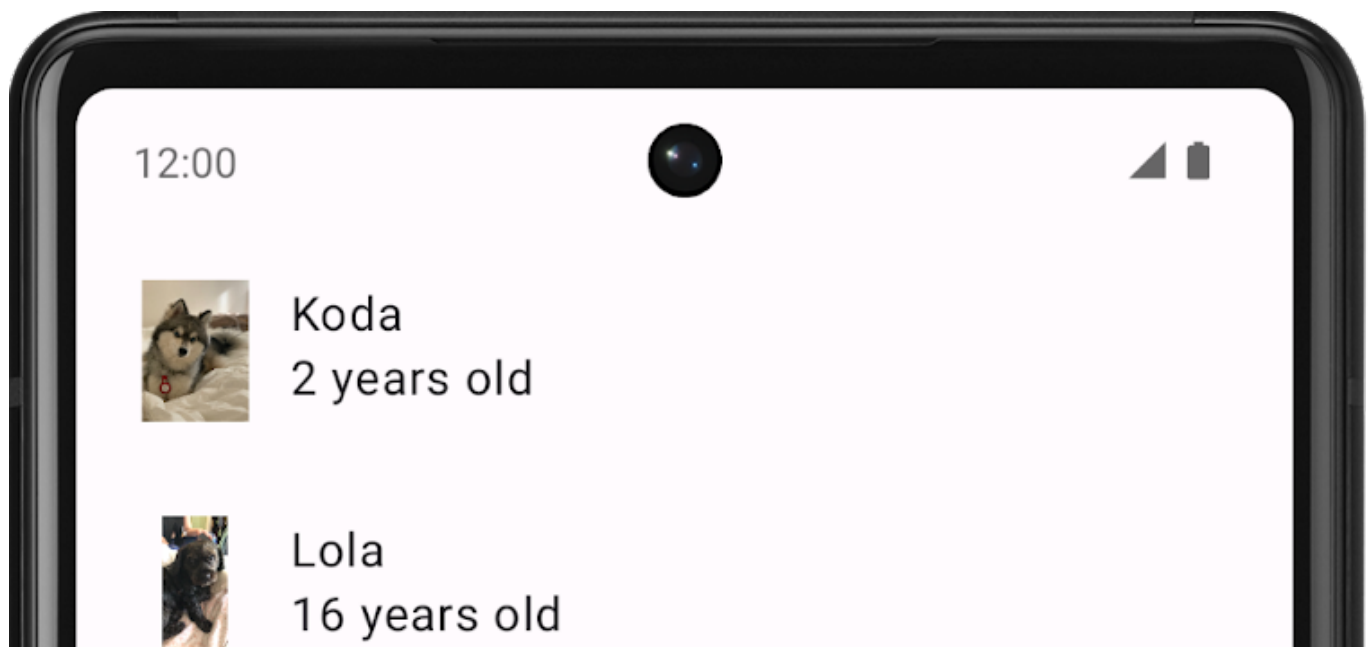
Например, если речь идет о кнопке, то один родитель может захотеть, чтобы его дочерняя кнопка использовала весь доступный размер, а другой родитель может захотеть, чтобы она обернула содержимое. Это делает код более многообразным. Этот параметр обычно называется «modifier» и должен быть первым необязательным параметром в списке параметров функции. **Этот модификатор применяется к первому дочернему методу.** Подробнее об этом читайте в руководстве по API для Jetpack Compose.

- Убедитесь, что на вашем эмуляторе/устройстве установлена светлая тема

В этом уроке вы будете работать как со светлой, так и с темной темой, однако большая часть урока будет посвящена светлой теме. Прежде чем приступить к работе, убедитесь, что ваше устройство/эмулятор находится в светлой теме.

Чтобы просмотреть ваше приложение в светлой теме, на эмуляторе или физическом устройстве:

- Зайдите в приложение «Настройки» на устройстве.
- Найдите пункт «Темная тема» и нажмите на него.
- Если темная тема включена, отключите ее.
- Запустите код запуска и посмотрите, с чего вы начнете: это список, в котором отображаются собаки с их фотографиями, именами и возрастом. Он функционален, но выглядит не очень, поэтому мы собираемся это исправить.





Frankie
2 years old



Nox
8 years old



Faye
8 years old



Bella
14 years old



Moana
2 years old



Tzeitel
7 years old



Leroy
4 years old



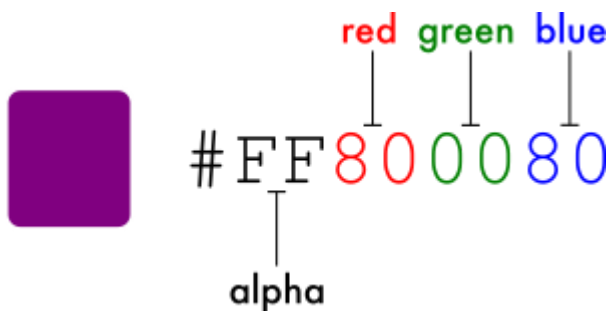
Добавьте цвет

Первое, что вам предстоит изменить в приложении Woof, - это цветовая схема.

Цветовая схема - это сочетание цветов, которые использует ваше приложение. Различные сочетания цветов вызывают разное настроение, что влияет на то, как люди чувствуют себя при использовании вашего приложения.

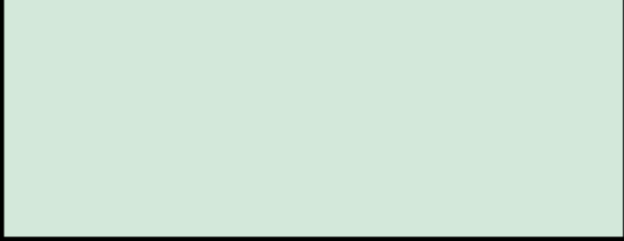
В системе Android цвет представлен шестнадцатеричным (hex) цветовым значением.

Шестнадцатеричный код цвета начинается с символа фунта (#), за которым следуют шесть букв и/или цифр, обозначающих красный, зеленый и синий (RGB) компоненты этого цвета. Первые две буквы/цифры относятся к красному цвету, следующие две - к зеленому, а последние две - к синему.



Цвет также может включать альфа-значение - буквы и/или цифры - которое отражает прозрачность цвета (#00 - 0% непрозрачности (полностью прозрачный), #FF - 100% непрозрачности (полностью непрозрачный)). Если альфа-значение включено, оно представляет собой первые два символа шестнадцатеричного кода цвета после знака фунта (#). Если альфа-значение не включено, считается, что оно равно #FF, то есть 100% непрозрачности (полностью непрозрачный).

Ниже приведены примеры цветов и их шестнадцатеричные значения.

Color	Hex color
	#006C4C
	#89F8C7
	#CFE9D9
	#4D6357

Используйте Material Theme Builder для создания цветовой схемы

Чтобы создать пользовательскую цветовую схему для нашего приложения, мы воспользуемся конструктором тем Material Theme Builder.

По адресу

<https://m3.material.io/theme-builder#/custom>

можно перейти в Material Theme Builder.

Только для ознакомления. В колледже не работает

На левой панели вы увидите основные цвета, нажмите на Primary:

Core colors

Input one or more brand color to define your color scheme.



Primary

Acts as custom source color



Secondary



Tertiary



Neutral

Used for background and surfaces

Откроется окно выбора цвета HCT.

HCT Color Picker



#6750A4

Hue

298.9809



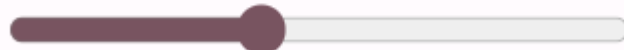
Chroma

47.85652



Tone

40.08324



Close

- Чтобы создать цветовую схему, показанную на скриншотах приложения, вы измените основной цвет в этом цветовом редакторе. В текстовом поле замените текущий текст на #006C4C. В результате основной цвет приложения станет зеленым.

HCT Color Picker



#006C4C

Hue

166.7024



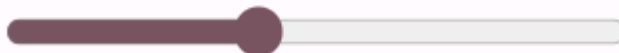
Chroma

40.59116



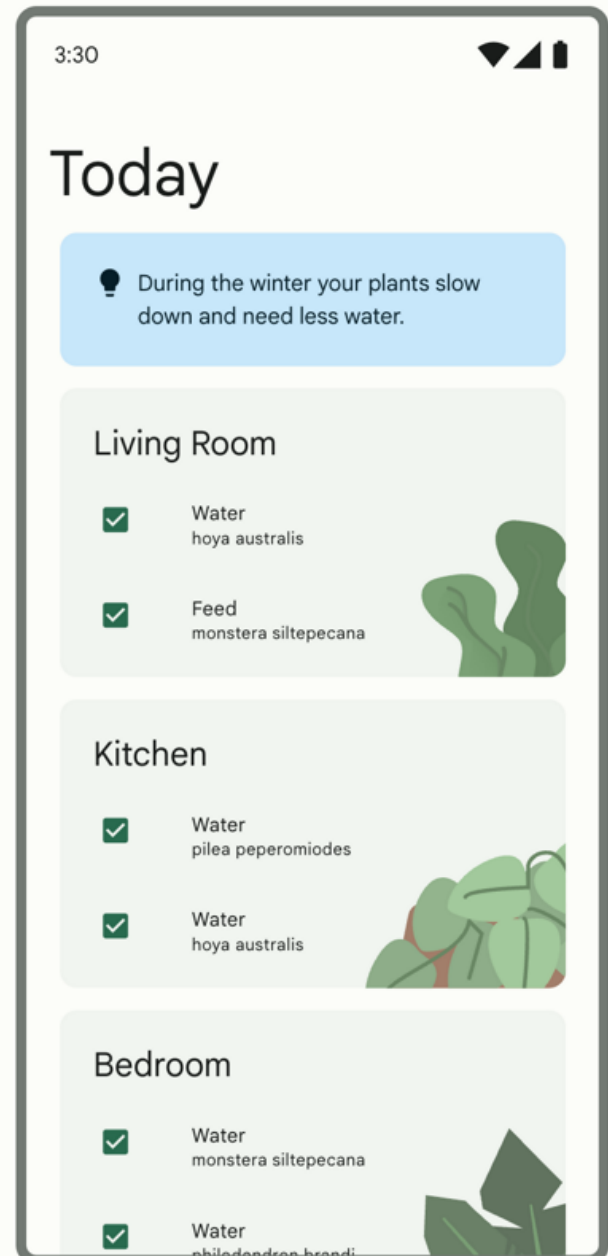
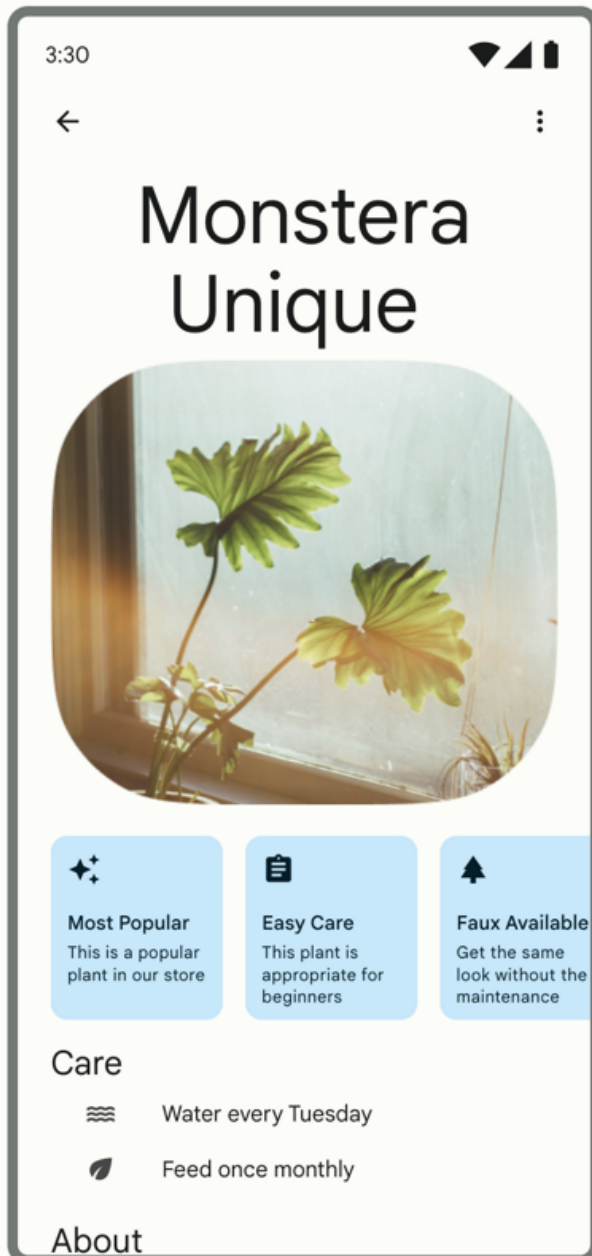
Tone

39.99328



Close

- Обратите внимание, как при этом приложения на экране приобретают зеленую цветовую гамму.



- Прокрутите страницу вниз, и вы увидите полную цветовую схему для светлой и темной темы, созданную на основе введенного вами цвета.

Light Scheme

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Dark Scheme

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Вам может быть интересно, что это за роли и как они используются, вот несколько основных из них:

- Основные цвета используются для ключевых компонентов пользовательского интерфейса.
- Вторичные цвета используются для менее заметных компонентов пользовательского интерфейса.
- Третичные цвета используются для контрастных акцентов, которые могут быть использованы для балансировки основного и вторичного цветов или привлечения повышенного внимания к элементу, например, к полю ввода.

Цветовые элементы появляются поверх других цветов в палитре и применяются в основном для текста, иконографии и штрихов. В нашей палитре цветов есть `onSurface color`, который появляется поверх цвета поверхности, и `onPrimary color`, который появляется поверх основного цвета.

Наличие этих слотов приводит к созданию целостной системы дизайна, в которой связанные компоненты окрашиваются одинаково.

Хватит теории о цветах, пора добавить эту красивую цветовую палитру в приложение!

Добавление цветовой палитры в тему

На странице Material Theme Builder есть возможность нажать кнопку **Export**, чтобы загрузить файл `Color.kt` и файл `Theme.kt` с пользовательской темой, созданной в Theme Builder.

- Это позволит добавить созданную нами пользовательскую тему в ваше приложение. Однако, поскольку сгенерированный файл `Theme.kt` не включает код для динамического цвета, который мы рассмотрим позже в уроке, скопируйте эти файлы.

Примечание: Если вы решите использовать файлы, сгенерированные в Material Theme Builder, для другого проекта, вам нужно будет обновить имя пакета до имени пакета вашего проекта.

Откройте файл `Color.kt` и замените его содержимое приведенным ниже кодом, чтобы скопировать новую цветовую схему.

```
package com.example.woof.ui.theme

import androidx.compose.ui.graphics.Color

val md_theme_light_primary = Color(0xFF006C4C)
val md_theme_light_onPrimary = Color(0xFFFFFFFF)
val md_theme_light_primaryContainer = Color(0xFF89F8C7)
val md_theme_light_onPrimaryContainer = Color(0xFF002114)
val md_theme_light_secondary = Color(0xFF4D6357)
val md_theme_light_onSecondary = Color(0xFFFFFFFF)
val md_theme_light_secondaryContainer = Color(0xFFCFE9D9)
val md_theme_light_onSecondaryContainer = Color(0xFF092016)
val md_theme_light_tertiary = Color(0xFF3D6373)
val md_theme_light_onTertiary = Color(0xFFFFFFFF)
val md_theme_light_tertiaryContainer = Color(0xFFC1E8FB)
val md_theme_light_onTertiaryContainer = Color(0xFF001F29)
val md_theme_light_error = Color(0xFFBA1A1A)
val md_theme_light_errorContainer = Color(0xFFFFDAD6)
val md_theme_light_onError = Color(0xFFFFFFFF)
val md_theme_light_onErrorContainer = Color(0xFF410002)
val md_theme_light_background = Color(0xFFFFBFD9)
val md_theme_light_onBackground = Color(0xFF191C1A)
val md_theme_light_surface = Color(0xFFFFBFD9)
val md_theme_light_onSurface = Color(0xFF191C1A)
val md_theme_light_surfaceVariant = Color(0xFFDBE5DD)
val md_theme_light_onSurfaceVariant = Color(0xFF404943)
val md_theme_light_outline = Color(0xFF707973)
val md_theme_light_inverseOnSurface = Color(0xFFEFF1ED)
val md_theme_light_inverseSurface = Color(0xFF2E312F)
val md_theme_light_inversePrimary = Color(0xFF6CDBAC)
val md_theme_light_shadow = Color(0xFF000000)
val md_theme_light_surfaceTint = Color(0xFF006C4C)
val md_theme_light_outlineVariant = Color(0xFFBFC9C2)
val md_theme_light_scrim = Color(0xFF000000)

val md_theme_dark_primary = Color(0xFF6CDBAC)
val md_theme_dark_onPrimary = Color(0xFF003826)
val md_theme_dark_primaryContainer = Color(0xFF005138)
val md_theme_dark_onPrimaryContainer = Color(0xFF89F8C7)
val md_theme_dark_secondary = Color(0xFFB3CCBE)
val md_theme_dark_onSecondary = Color(0xFF1F352A)
val md_theme_dark_secondaryContainer = Color(0xFF354B40)
```

```

val md_theme_dark_onSecondaryContainer = Color(0xFFCFE9D9)
val md_theme_dark_tertiary = Color(0xFFA5CCDF)
val md_theme_dark_onTertiary = Color(0xFF073543)
val md_theme_dark_tertiaryContainer = Color(0xFF244C5B)
val md_theme_dark_onTertiaryContainer = Color(0xFFC1E8FB)
val md_theme_dark_error = Color(0xFFFFB4AB)
val md_theme_dark_errorContainer = Color(0xFF93000A)
val md_theme_dark_onError = Color(0xFF690005)
val md_theme_dark_onErrorContainer = Color(0xFFFFDAD6)
val md_theme_dark_background = Color(0xFF191C1A)
val md_theme_dark_onBackground = Color(0xFFE1E3DF)
val md_theme_dark_surface = Color(0xFF191C1A)
val md_theme_dark_onSurface = Color(0xFFE1E3DF)
val md_theme_dark_surfaceVariant = Color(0xFF404943)
val md_theme_dark_onSurfaceVariant = Color(0xFFBFC9C2)
val md_theme_dark_outline = Color(0xFF8A938C)
val md_theme_dark_inverseOnSurface = Color(0xFF191C1A)
val md_theme_dark_inverseSurface = Color(0xFFE1E3DF)
val md_theme_dark_inversePrimary = Color(0xFF006C4C)
val md_theme_dark_shadow = Color(0xFF000000)
val md_theme_dark_surfaceTint = Color(0xFF6CDBAC)
val md_theme_dark_outlineVariant = Color(0xFF404943)
val md_theme_dark_scrim = Color(0xFF000000)

```

- Откройте файл `Theme.kt` и замените его содержимое на приведенный ниже код, чтобы добавить новые цвета в тему.

```

package com.example.woof.ui.theme

import android.app.Activity
import android.os.Build
import android.view.View
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.SideEffect
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalView
import androidx.core.view.WindowCompat

private val LightColors = lightColorScheme(
    primary = md_theme_light_primary,
    onPrimary = md_theme_light_onPrimary,
    primaryContainer = md_theme_light_primaryContainer,
    onPrimaryContainer = md_theme_light_onPrimaryContainer,

```

```

        secondary = md_theme_light_secondary,
        onSecondary = md_theme_light_onSecondary,
        secondaryContainer = md_theme_light_secondaryContainer,
        onSecondaryContainer = md_theme_light_onSecondaryContainer,
        tertiary = md_theme_light_tertiary,
        onTertiary = md_theme_light_onTertiary,
        tertiaryContainer = md_theme_light_tertiaryContainer,
        onTertiaryContainer = md_theme_light_onTertiaryContainer,
        error = md_theme_light_error,
        errorContainer = md_theme_light_errorContainer,
        onError = md_theme_light_onError,
        onErrorContainer = md_theme_light_onErrorContainer,
        background = md_theme_light_background,
        onBackground = md_theme_light_onBackground,
        surface = md_theme_light_surface,
        onSurface = md_theme_light_onSurface,
        surfaceVariant = md_theme_light_surfaceVariant,
        onSurfaceVariant = md_theme_light_onSurfaceVariant,
        outline = md_theme_light_outline,
        inverseOnSurface = md_theme_light_inverseOnSurface,
        inverseSurface = md_theme_light_inverseSurface,
        inversePrimary = md_theme_light_inversePrimary,
        surfaceTint = md_theme_light_surfaceTint,
        outlineVariant = md_theme_light_outlineVariant,
        scrim = md_theme_light_scrim,
    )

```

```

private val DarkColors = darkColorScheme(
    primary = md_theme_dark_primary,
    onPrimary = md_theme_dark_onPrimary,
    primaryContainer = md_theme_dark_primaryContainer,
    onPrimaryContainer = md_theme_dark_onPrimaryContainer,
    secondary = md_theme_dark_secondary,
    onSecondary = md_theme_dark_onSecondary,
    secondaryContainer = md_theme_dark_secondaryContainer,
    onSecondaryContainer = md_theme_dark_onSecondaryContainer,
    tertiary = md_theme_dark_tertiary,
    onTertiary = md_theme_dark_onTertiary,
    tertiaryContainer = md_theme_dark_tertiaryContainer,
    onTertiaryContainer = md_theme_dark_onTertiaryContainer,
    error = md_theme_dark_error,
    errorContainer = md_theme_dark_errorContainer,
    onError = md_theme_dark_onError,
    onErrorContainer = md_theme_dark_onErrorContainer,
    background = md_theme_dark_background,
    onBackground = md_theme_dark_onBackground,
    surface = md_theme_dark_surface,
    onSurface = md_theme_dark_onSurface,
    surfaceVariant = md_theme_dark_surfaceVariant,
    onSurfaceVariant = md_theme_dark_onSurfaceVariant,
    outline = md_theme_dark_outline,
    inverseOnSurface = md_theme_dark_inverseOnSurface,
    inverseSurface = md_theme_dark_inverseSurface,
)

```

```

        inversePrimary = md_theme_dark_inversePrimary,
        surfaceTint = md_theme_dark_surfaceTint,
        outlineVariant = md_theme_dark_outlineVariant,
        scrim = md_theme_dark_scrim,
    )

    @Composable
    fun WoofTheme(
        darkTheme: Boolean = isSystemInDarkTheme(),
        // Dynamic color is available on Android 12+
        dynamicColor: Boolean = false,
        content: @Composable () -> Unit
    ) {
        val colorScheme = when {
            dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
                val context = LocalContext.current
                if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
            }

            darkTheme -> DarkColors
            else -> LightColors
        }
        val view = LocalView.current
        if (!view.isInEditMode) {
            SideEffect {
                setUpEdgeToEdge(view, darkTheme)
            }
        }

        MaterialTheme(
            colorScheme = colorScheme,
            shapes = Shapes,
            typography = Typography,
            content = content
        )
    }

    /**
     * Устанавливает от края до края для окна этого [вида]. Цвета системных значков
     * устанавливаются либо
     * светлыми или темными в зависимости от того, включена ли тема [darkTheme] или
     * нет.
     */
    private fun setUpEdgeToEdge(view: View, darkTheme: Boolean) {
        val window = (view.context as Activity).window
        WindowCompat.setDecorFitsSystemWindows(window, false)
        window.statusBarColor = Color.Transparent.toArgb()
        val navigationBarColor = when {
            Build.VERSION.SDK_INT >= 29 -> Color.Transparent.toArgb()
            Build.VERSION.SDK_INT >= 26 -> Color(0xFF, 0xFF, 0xFF, 0x63).toArgb()
            // Min sdk version for this app is 24, this block is for SDK versions 24
and 25
            else -> Color(0x00, 0x00, 0x00, 0x50).toArgb()
        }
    }

```

```

    }
    window.navigationBarColor = navigationBarColor
    val controller = WindowCompat.getInsetsController(window, view)
    controller.isAppearanceLightStatusBars = !darkTheme
    controller.isAppearanceLightNavigationBars = !darkTheme
}

```

В `WoofTheme()` в `val colorScheme` используется оператор `when`.

Если `dynamicColor` равен `true` и версия сборки `S` или выше, проверяется, находится ли устройство в темной теме или нет. Если оно находится в темной теме, `colorScheme` будет установлен в `dynamicDarkColorScheme`. Если темной темы нет, будет установлено значение `dynamicLightColorScheme`.

Если приложение не использует `dynamicColorScheme`, проверяется, находится ли ваше приложение в темной теме. Если да, то для `colorScheme` будет установлено значение `DarkColors`. Если ни то, ни другое не верно, то `colorScheme` будет установлен в `LightColors`. В скопированном в файл `Theme.kt` файле `dynamicColor` установлено значение `false`, а устройства, с которыми мы работаем, находятся в светлом режиме, поэтому `colorScheme` будет установлен в `LightColors`.

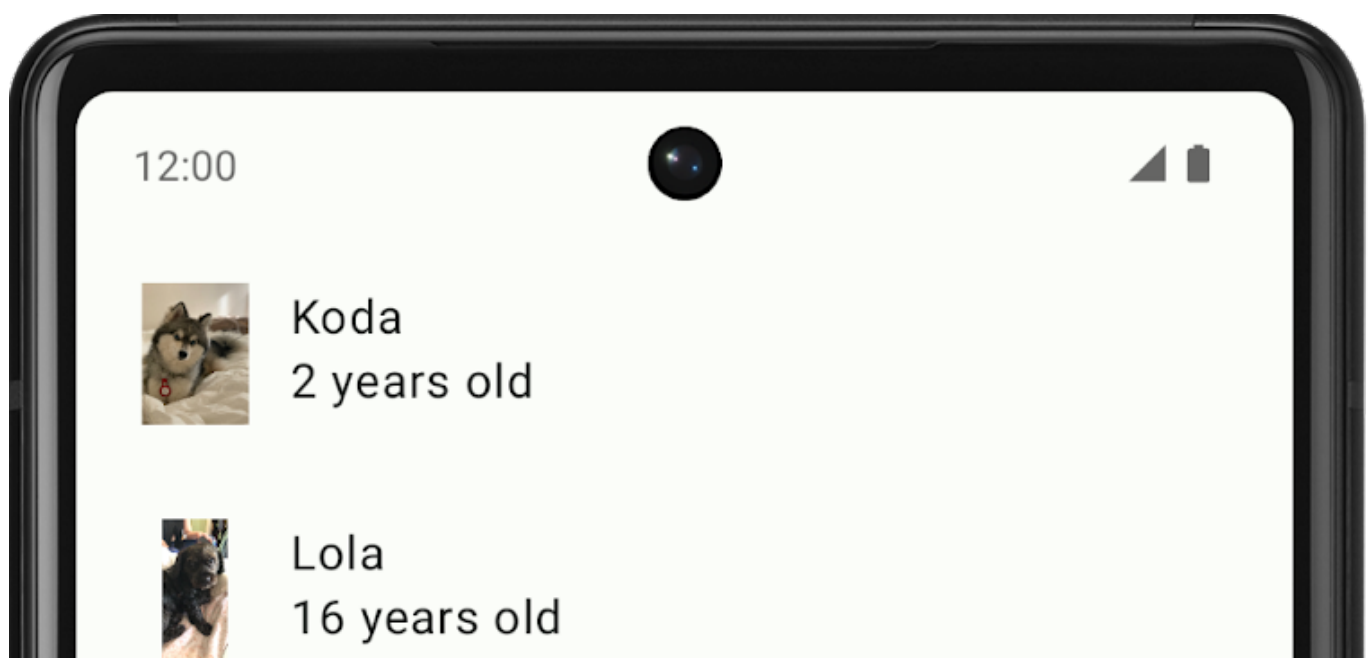
```

val colorScheme = when {
    dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
        val context = LocalContext.current
        if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
    }

    darkTheme -> DarkColors
    else -> LightColors
}

```

- Запустите приложение снова, обратите внимание, что панель приложений автоматически изменила цвет.





Frankie
2 years old



Nox
8 years old



Faye
8 years old



Bella
14 years old



Moana
2 years old



Tzeitel
7 years old



Leroy
4 years old



Сопоставление цветов

Компоненты материалов автоматически привязываются к цветовым слотам. Другие ключевые компоненты пользовательского интерфейса, такие как плавающие кнопки действий, также по умолчанию используют основной цвет. Это означает, что вам не нужно явно назначать цвет компоненту; он автоматически сопоставляется с цветовым слотом, когда вы устанавливаете цветовую тему в приложении. Вы можете отменить это, явно задав цвет в коде.

В этом разделе мы обернем строку, содержащую `DogIcon()` и `DogInformation()`, карточкой, чтобы отличить цвет элементов списка от фона.

В композитной функции `DogItem()` оберните `Row()` с помощью `Card()`.

```
Card() {  
    Row(  
        modifier = modifier  
            .fillMaxWidth()  
            .padding(dimensionResource(id = R.dimen.padding_small))  
    ) {  
        DogIcon(dog.imageResourceId)  
        DogInformation(dog.name, dog.age)  
    }  
}
```

Поскольку `Card` теперь является первым дочерним объектом, компонуемым в `DogItem()`, передайте модификатор из `DogItem()` в `Card` и обновите модификатор `Row` на новый экземпляр `Modifier`.

```
Card(modifier = modifier) {  
    Row(  
        modifier = Modifier  
            .fillMaxWidth()  
            .padding(dimensionResource(id = R.dimen.padding_small))  
    ) {  
        DogIcon(dog.imageResourceId)  
        DogInformation(dog.name, dog.age)  
    }  
}
```

Взгляните на функцию `WoofPreview()`. Теперь элементы списка автоматически меняют цвет благодаря `Card Composables`. Цвета выглядят отлично, но между элементами списка нет интервала.

WoofPreview



Koda
2 years old



Lola
16 years old



Frankie
2 years old



Nox
8 years old



Faye
8 years old



Bella
14 years old



Moana
2 years old



Tzeitel
7 years old



Leroy
4 years old

Файл `dimens` Точно так же, как вы используете файл `strings.xml` для хранения строк в вашем приложении, рекомендуется использовать файл `dimens.xml` для хранения значений размеров. Это

полезно для того, чтобы не кодировать значения жестко, и чтобы в случае необходимости их можно было изменить в одном месте.

Перейдите в `app > res > values > dims.xml` и посмотрите на файл. В нем хранятся значения размеров `padding_small`, `padding_medium` и `image_size`. Эти размеры будут использоваться во всем приложении.

```
<resources>
  <dimen name="padding_small">8dp</dimen>
  <dimen name="padding_medium">16dp</dimen>
  <dimen name="image_size">64dp</dimen>
</resources>
```

- Чтобы добавить значение из файла `dims.xml`, воспользуйтесь этим правильным форматом:

`dimensionResource(id = R.id. add dimension name here )`

Например, чтобы добавить `padding_small`, вы передадите `dimensionResource(id = R.dimen.padding_small)`.

В `WoofApp()` добавьте модификатор с `padding_small` в вызов `DogItem()`.

```
@Composable
fun WoofApp() {
    Scaffold { it ->
        LazyColumn(contentPadding = it) {
            items(dogs) {
                DogItem(
                    dog = it,
                    modifier =
Modifier.padding(dimensionResource(R.dimen.padding_small))
                )
            }
        }
    }
}
```

В функции `WoofPreview()` теперь больше расстояния между элементами списка.

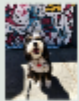
WoofPreview



Koda
2 years old



Lola
16 years old



Frankie
2 years old



Nox
8 years old



Faye
8 years old



Bella
14 years old



Moana
2 years old



Tzeitel
7 years old



Leroy
4 years old

В системе Android есть возможность переключить устройство на темную тему. В темной теме используются более темные и приглушенные цвета:

- Может значительно снизить энергопотребление (в зависимости от технологии экрана устройства).
- Улучшает видимость для пользователей со слабым зрением и тех, кто чувствителен к яркому свету.
- Упрощает использование устройства в условиях недостаточной освещенности.

Ваше приложение может выбрать Force Dark, что означает, что система будет реализовывать темную тему для вас. Однако для пользователей будет лучше, если вы сами внедрите темную тему, чтобы сохранить полный контроль над темой приложения.

При выборе темной темы важно учитывать, что цвета для темной темы должны соответствовать стандартам контрастности. В темных темах используется темный цвет поверхности с ограниченным количеством цветовых акцентов.

Просмотр темной темы в предварительном просмотре

Вы уже добавили цвета для темной темы в предыдущем шаге. Чтобы увидеть темную тему в действии, вы добавите еще один компонент Preview Composable в MainActivity.kt. Таким образом, когда вы измените макет пользовательского интерфейса в коде, вы сможете увидеть, как одновременно выглядят превью светлой и темной тем.

- В разделе WoofPreview() создайте новую функцию WoofDarkThemePreview() и аннотируйте ее с помощью @Preview и @Composable.

```
@Preview
@Composable
fun WoofDarkThemePreview() {

}
```

- Внутри DarkThemePreview() добавьте WoofTheme(). Без добавления WoofTheme() вы не увидите ни одного из стилей, которые мы добавили в приложение. Установите для параметра darkTheme значение true.

```
@Preview
@Composable
fun WoofDarkThemePreview() {
    WoofTheme(darkTheme = true) {

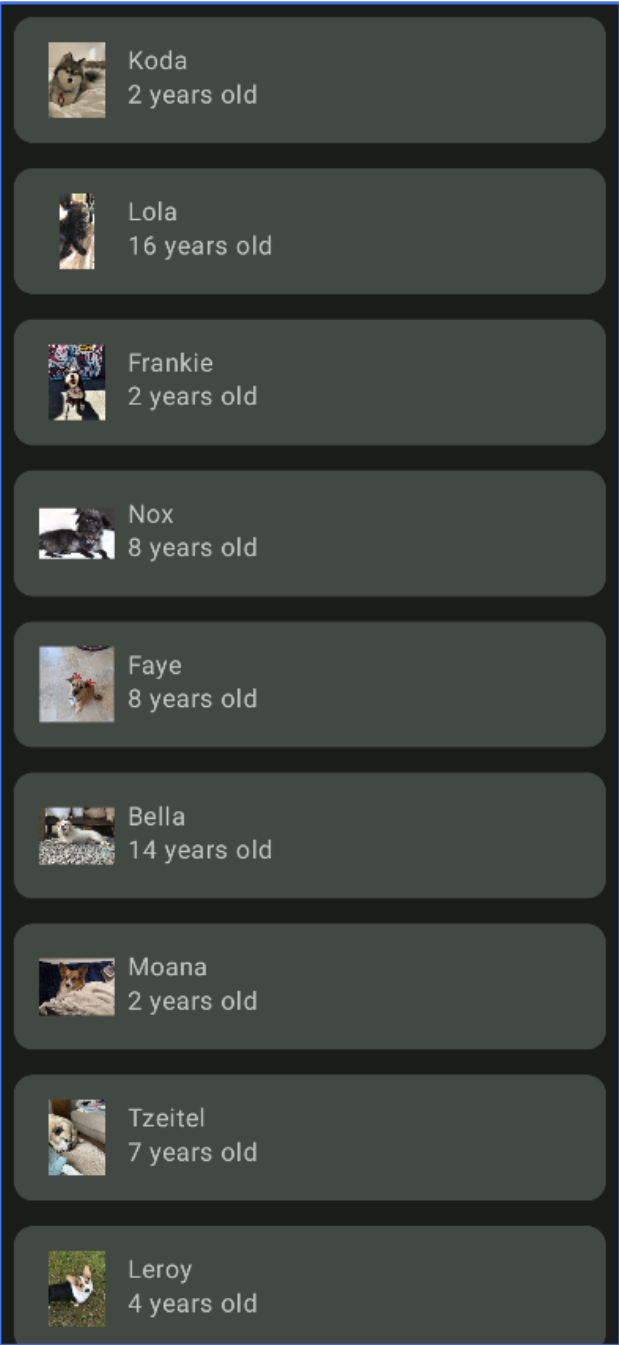
    }
}
```

- Вызовите WoofApp() внутри WoofTheme().

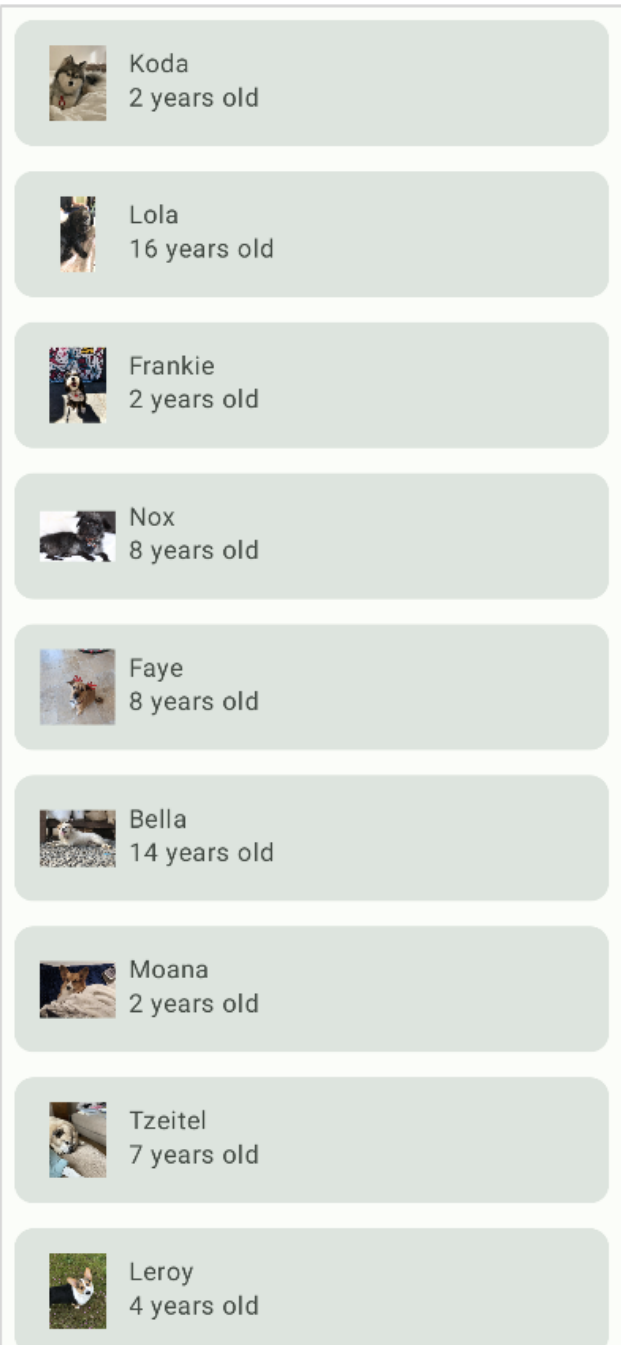
```
@Preview
@Composable
fun WoofDarkThemePreview() {
    WoofTheme(darkTheme = true) {
        WoofApp()
    }
}
```

Теперь прокрутите панель «Дизайн» вниз, чтобы увидеть приложение в темной теме, включая более темный фон приложения/элемента списка и более светлый текст. Сравните различия между темной и светлой темами.

WoofDarkThemePreview



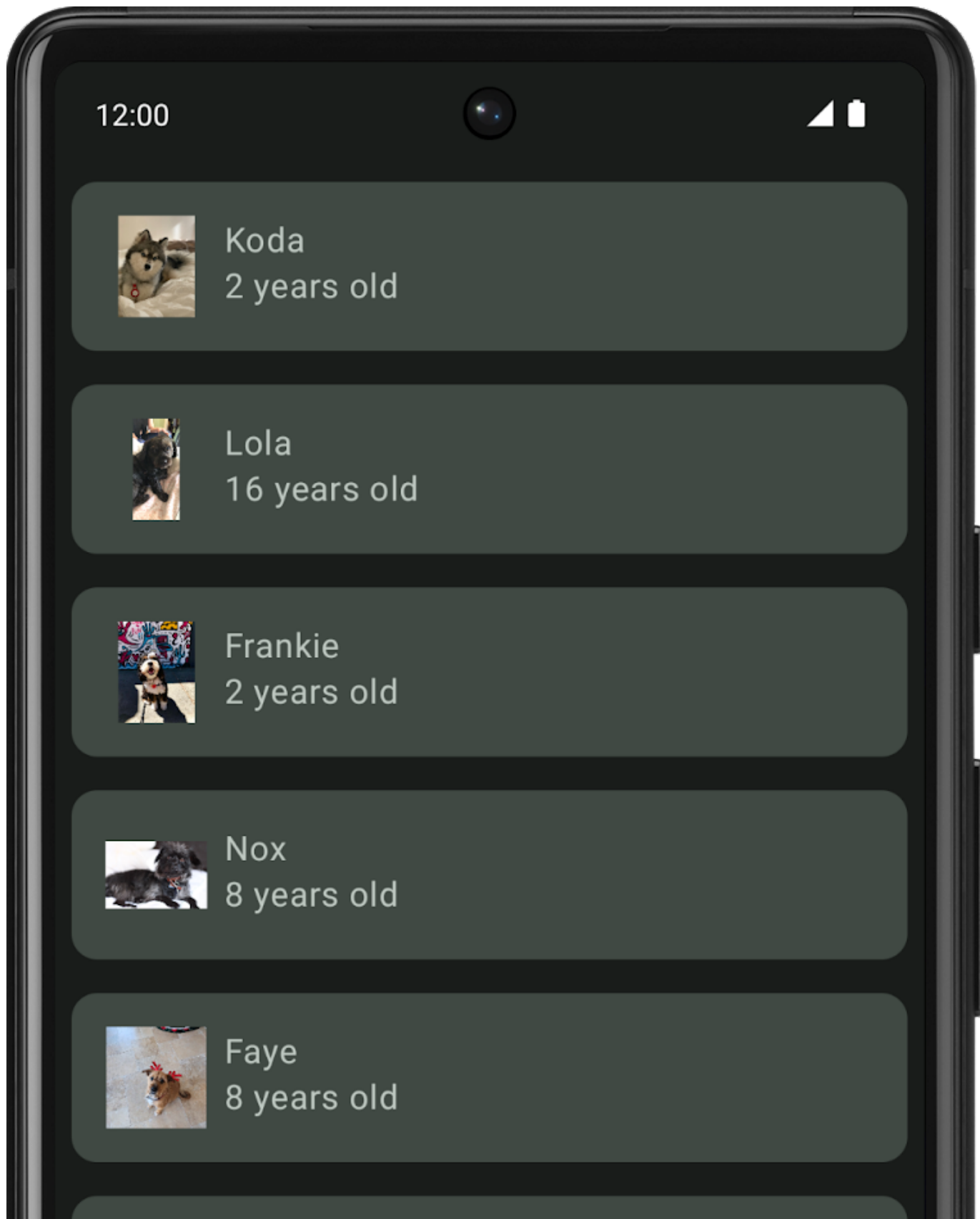
WoofPreview

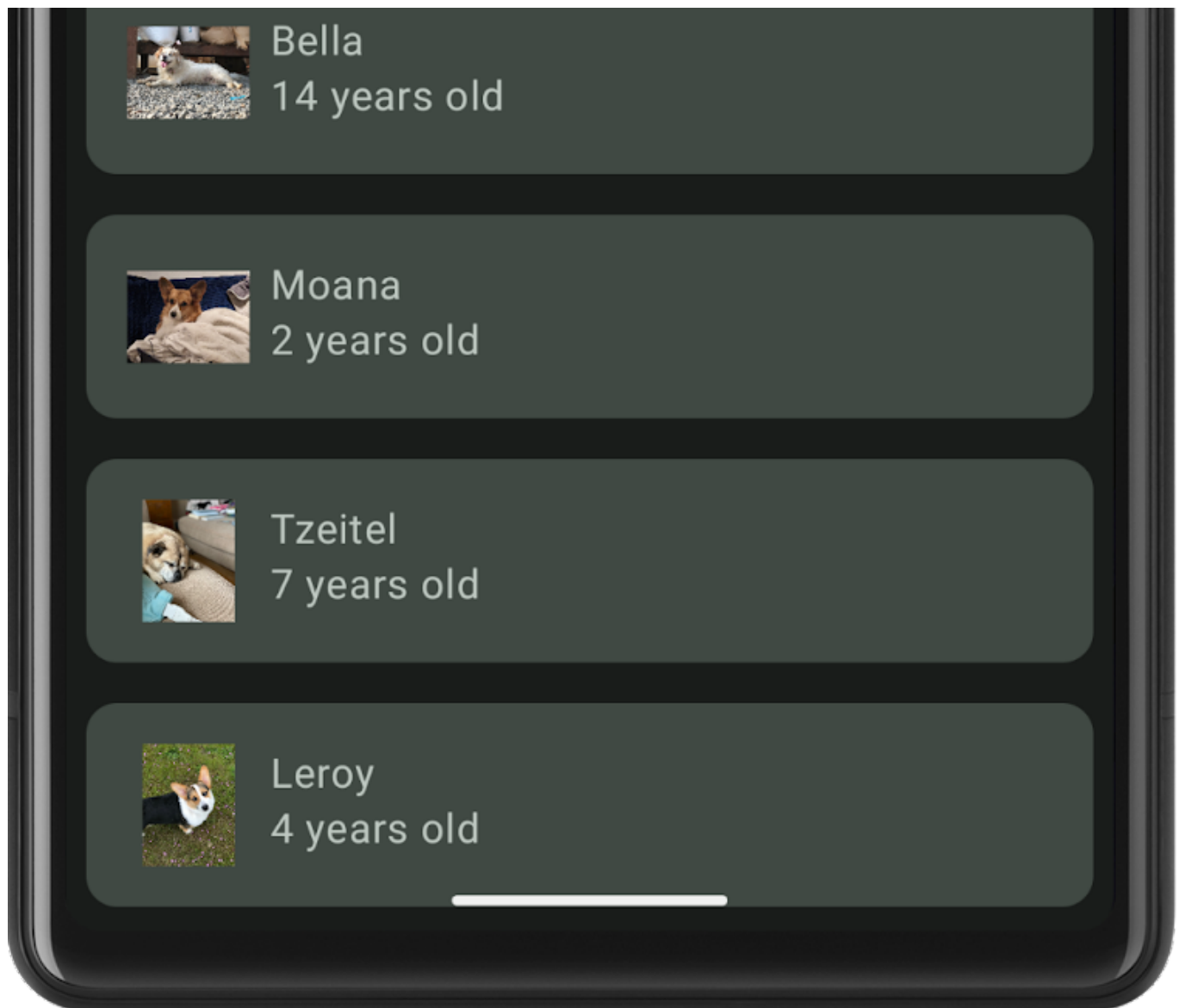


Просмотр темной темы на устройстве или эмуляторе

Чтобы просмотреть свое приложение в темной теме на эмуляторе или физическом устройстве:

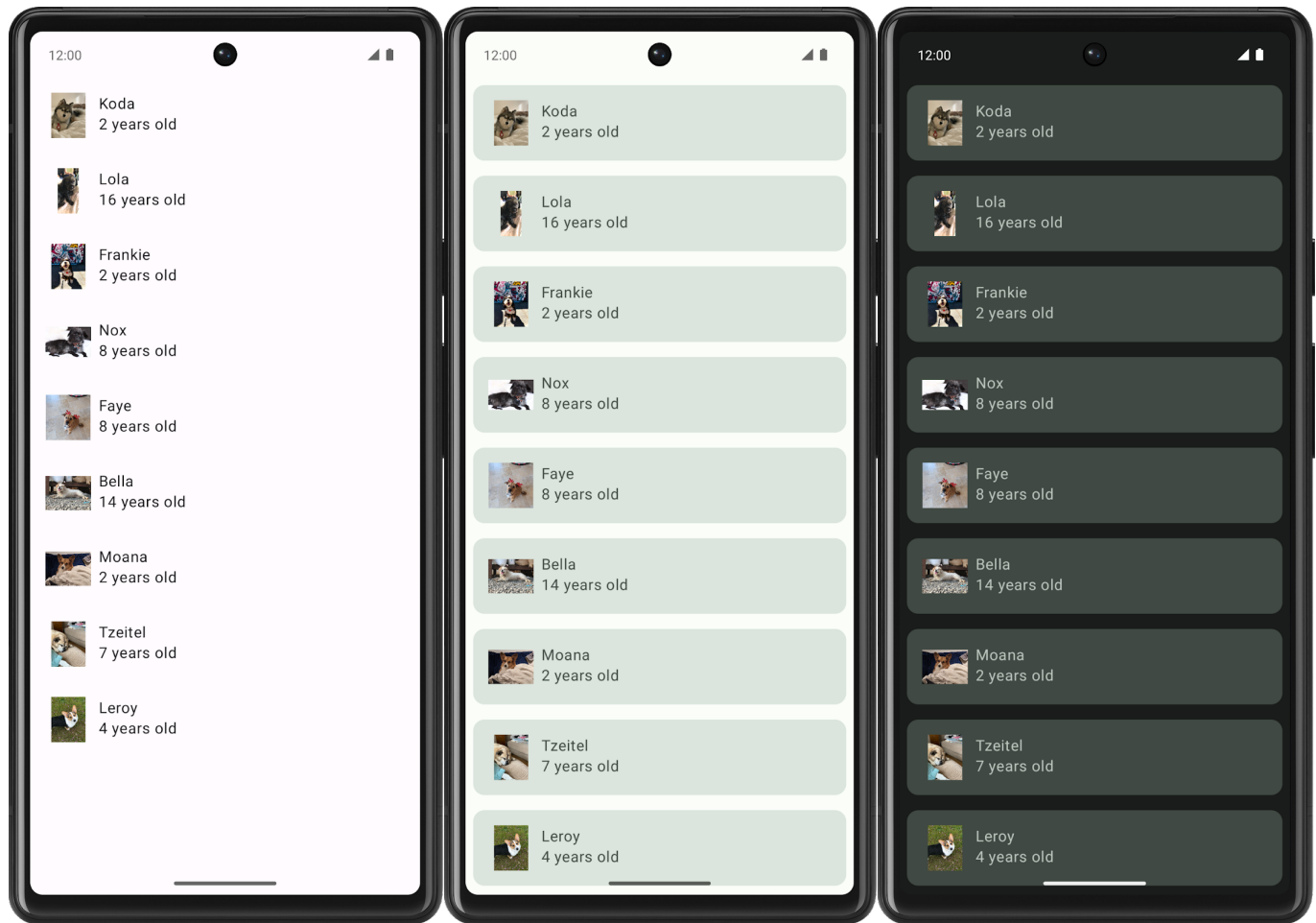
- Зайдите в приложение «Настройки» на устройстве.
- Найдите пункт «Темная тема» и нажмите на него.
- Включите темную тему.
- Снова откройте приложение Woof, и оно перейдет в темную тему.





Этот урκο больше сосредоточен на светлой теме, поэтому перед тем, как приступить к работе с приложением, отключите темную тему.

- Перейдите в приложение «Настройки» на устройстве.
- Выберите Дисплей.
- Отключите темную тему.
- Сравните, как выглядело приложение в начале раздела и сейчас. Элементы списка и текст стали более четкими, а цветовая схема - более привлекательной.



Динамический цвет

В Material 3 большое внимание уделяется персонализации пользователей - новая функция в Material 3 - Dynamic Color, которая создает тему для вашего приложения на основе обоев пользователя. Таким образом, если пользователь любит зеленый цвет и имеет синий фон телефона, его приложение Woof также будет синим, чтобы отразить это. Динамическая тема работает только на некоторых устройствах под управлением Android 12 и выше.

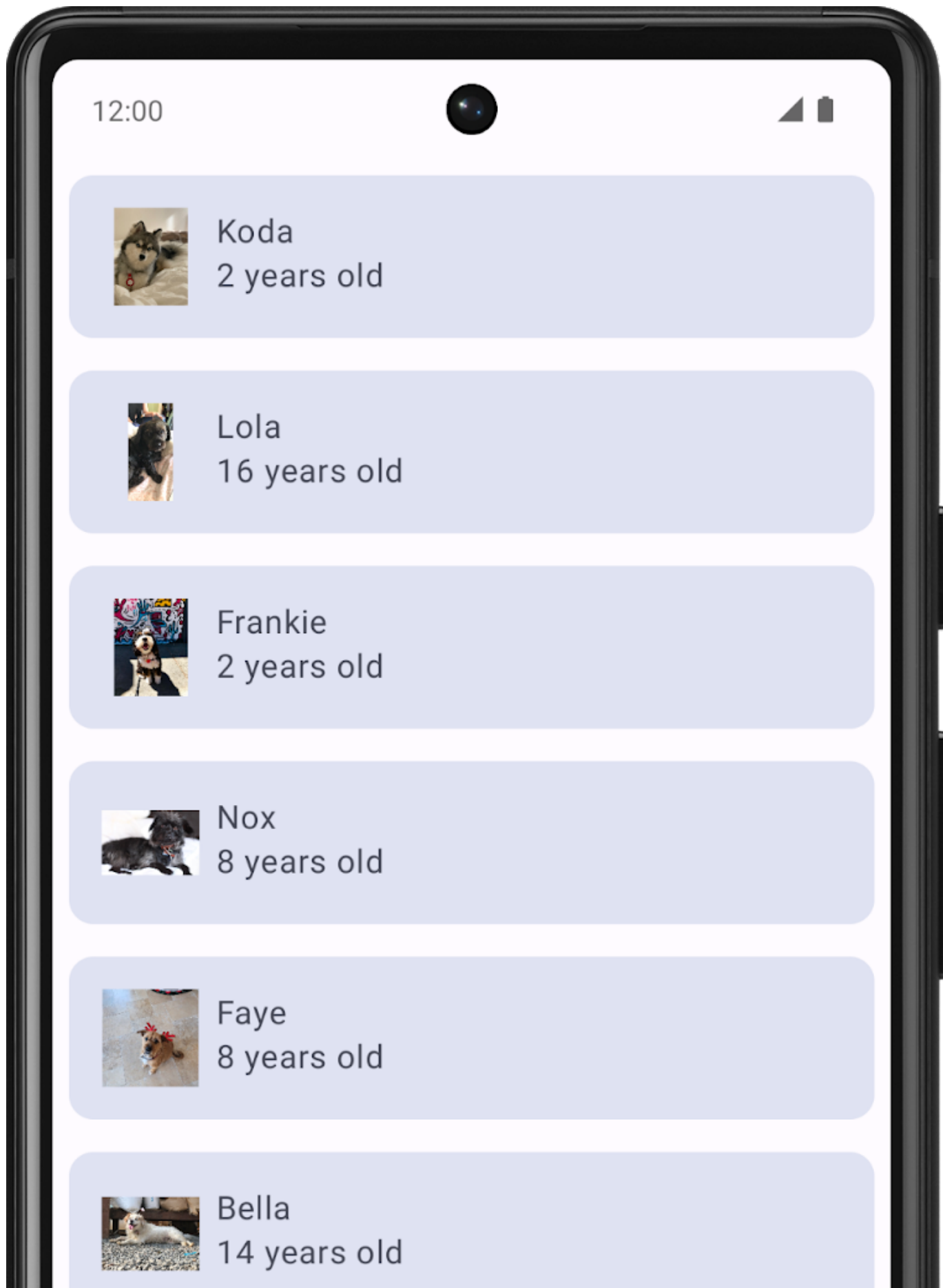
Пользовательская тема может быть использована для приложений, которые имеют сильные брендинговые цвета, а также должна быть реализована для устройств, которые не поддерживают динамическую тематику, чтобы ваше приложение оставалось тематическим.

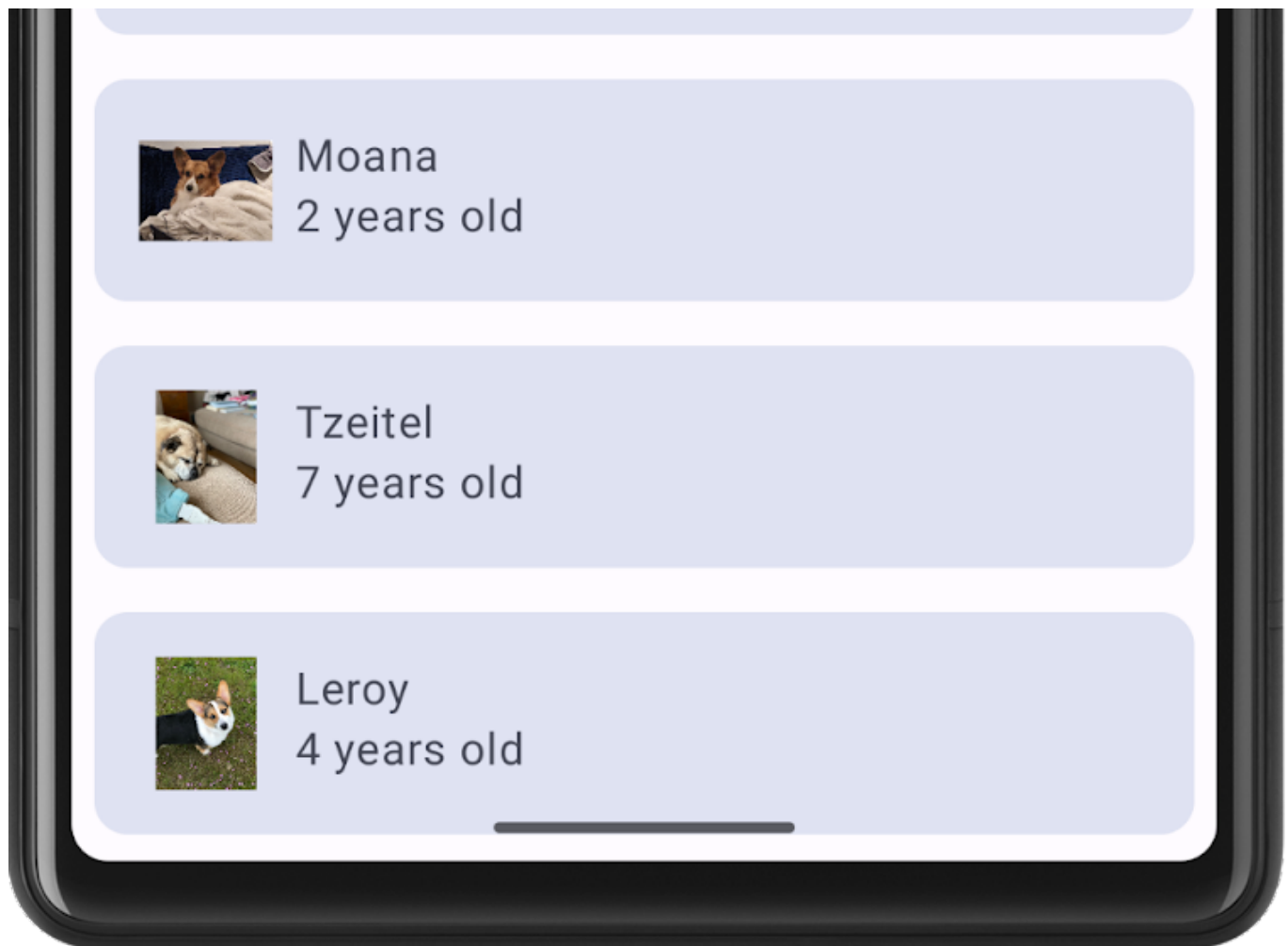
Чтобы включить динамический цвет, откройте [Theme.kt](#), перейдите к композиту `WoofTheme()` и установите параметр `dynamicColor` в `true`.

```
@Composable
fun WoofTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
)
```

- Чтобы изменить фон устройства или эмулятора, зайдите в «Настройки», затем найдите «Обои».

- Измените обои на цвет или набор цветов.
- Перезапустите приложение, чтобы увидеть динамическую тему (обратите внимание, что ваше устройство или эмулятор должны быть на Android 12+, чтобы увидеть динамический цвет)





В этом уроке рассматриваются пользовательские темы, поэтому отключите `dynamicColor`, прежде чем двигаться дальше.

```
@Composable
fun WoofTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
)
```

Добавьте форму

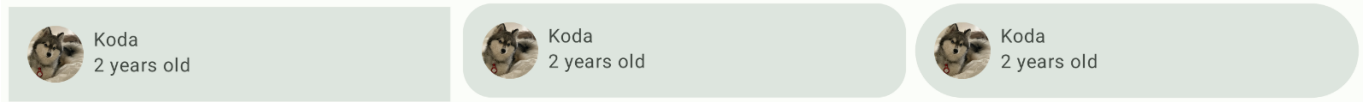
Применение формы может изменить очень многое во внешнем виде и восприятии композита. Формы направляют внимание, идентифицируют компоненты, передают состояние и выражают бренд.

Многие формы определяются с помощью `RoundedCornerShape`, которая описывает прямоугольник с закругленными углами. Передаваемое число определяет, насколько закруглены углы. Если используется `RoundedCornerShape(0.dp)`, прямоугольник не имеет закругленных углов; если используется `RoundedCornerShape(50.dp)`, углы будут полностью круглыми.

0.dp

25.dp

50.dp



Вы также можете изменять формы, добавляя различные проценты округления на каждом углу.

Top left: 50.dp

Top left: 15.dp

Top left: 0.dp

Bottom left: 25.dp

Bottom left: 50.dp

Bottom left: 50.dp

Top right: 0.dp

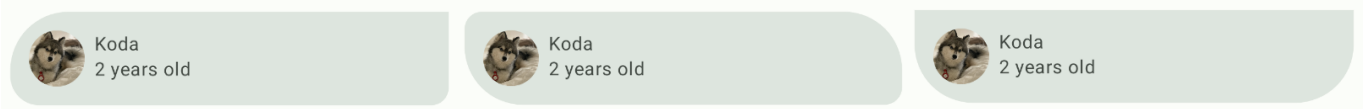
Top right: 50.dp

Top right: 0.dp

Bottom right: 15.dp

Bottom right: 15.dp

Bottom right: 50.dp



Файл **Shape.kt** используется для определения форм компонентов в Compose. Существует три типа компонентов: маленькие, средние и большие. В этом разделе вы измените компонент Card, который определен как средний. Компоненты группируются в категории форм в зависимости от их размера.

В этом разделе вы придадите изображению собаки форму круга и измените форму элемента списка.

Придайте изображению собаки форму круга

- Откройте файл **Shape.kt** и обратите внимание, что для параметра `small` установлено значение **`RoundedCornerShape(50.dp)`**. Этот параметр будет использоваться для придания изображению формы круга.

```
val Shapes = Shapes(
    small = RoundedCornerShape(50.dp),
)
```

- Откройте файл **MainActivity.kt**. В функции **DogIcon()** добавьте атрибут `clip` к модификатору **Image**; это позволит закрепить изображение в форме.
- Передайте материал **MaterialTheme.shapes.small**.

```
import androidx.compose.ui.draw.clip

@Composable
fun DogIcon(
    @DrawableRes dogIcon: Int,
    modifier: Modifier = Modifier
) {
    Image(
        modifier = modifier
            .size(dimensionResource(id = R.dimen.image_size))
    )
}
```

```
.padding(dimensionResource(id = R.dimen.padding_small))  
.clip(MaterialTheme.shapes.small),
```

- Когда вы посмотрите на `WoofPreview()`, вы заметите, что иконки собак круглые! Однако некоторые фотографии обрезаны по бокам и не отображаются полностью. `cular`.

WoofPreview



Koda
2 years old



Lola
16 years old



Frankie
2 years old



Nox
8 years old



Faye
8 years old



Bella
14 years old



Moana
2 years old



Tzeitel
7 years old



Leroy
4 years old

Чтобы сделать все фотографии круглыми, добавьте `ContentScale` и атрибут `Crop`; это обрезает изображение по размеру. Обратите внимание, что `contentScale` - это атрибут `Image`, а не часть модификатора.

```
import androidx.compose.ui.layout.ContentScale

@Composable
fun DogIcon(
    @DrawableRes dogIcon: Int,
    modifier: Modifier = Modifier
) {
    Image(
        modifier = modifier
            .size(dimensionResource(id = R.dimen.image_size))
            .padding(dimensionResource(id = R.dimen.padding_small))
            .clip(MaterialTheme.shapes.small),
        contentScale = ContentScale.Crop,
```

Это полная версия `DogIcon()` Composable.

```
@Composable
fun DogIcon(
    @DrawableRes dogIcon: Int,
    modifier: Modifier = Modifier
) {
    Image(
        modifier = modifier
            .size(dimensionResource(R.dimen.image_size))
            .padding(dimensionResource(R.dimen.padding_small))
            .clip(MaterialTheme.shapes.small),
        contentScale = ContentScale.Crop,
        painter = painterResource(dogIcon),

        // Content Description is not needed here - image is decorative, and
        // setting a null content
        // description allows accessibility services to skip this element during
        // navigation.

        contentDescription = null
    )
}
```

Теперь в `WoofPreview()` иконки стали круглыми.

WoofPreview



Koda
2 years old



Lola
16 years old



Frankie
2 years old



Nox
8 years old



Faye
8 years old



Bella
14 years old



Moana
2 years old



Tzeitel
7 years old

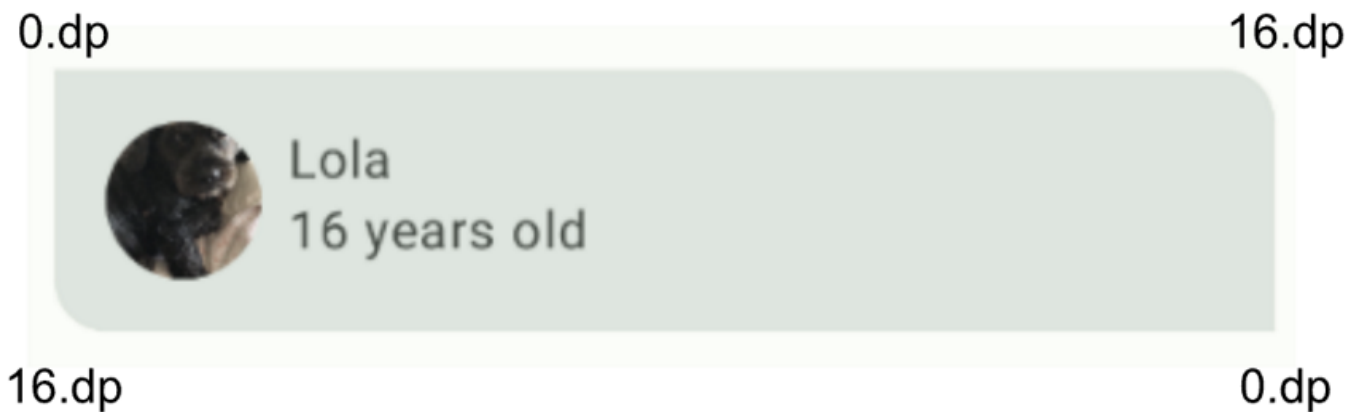


Leroy
4 years old

Добавление фигуры к элементу списка

В этом разделе вы добавите форму к элементу списка. Элемент списка уже отображается через карточку. Карточка - это поверхность, которая может содержать один составной элемент и содержит

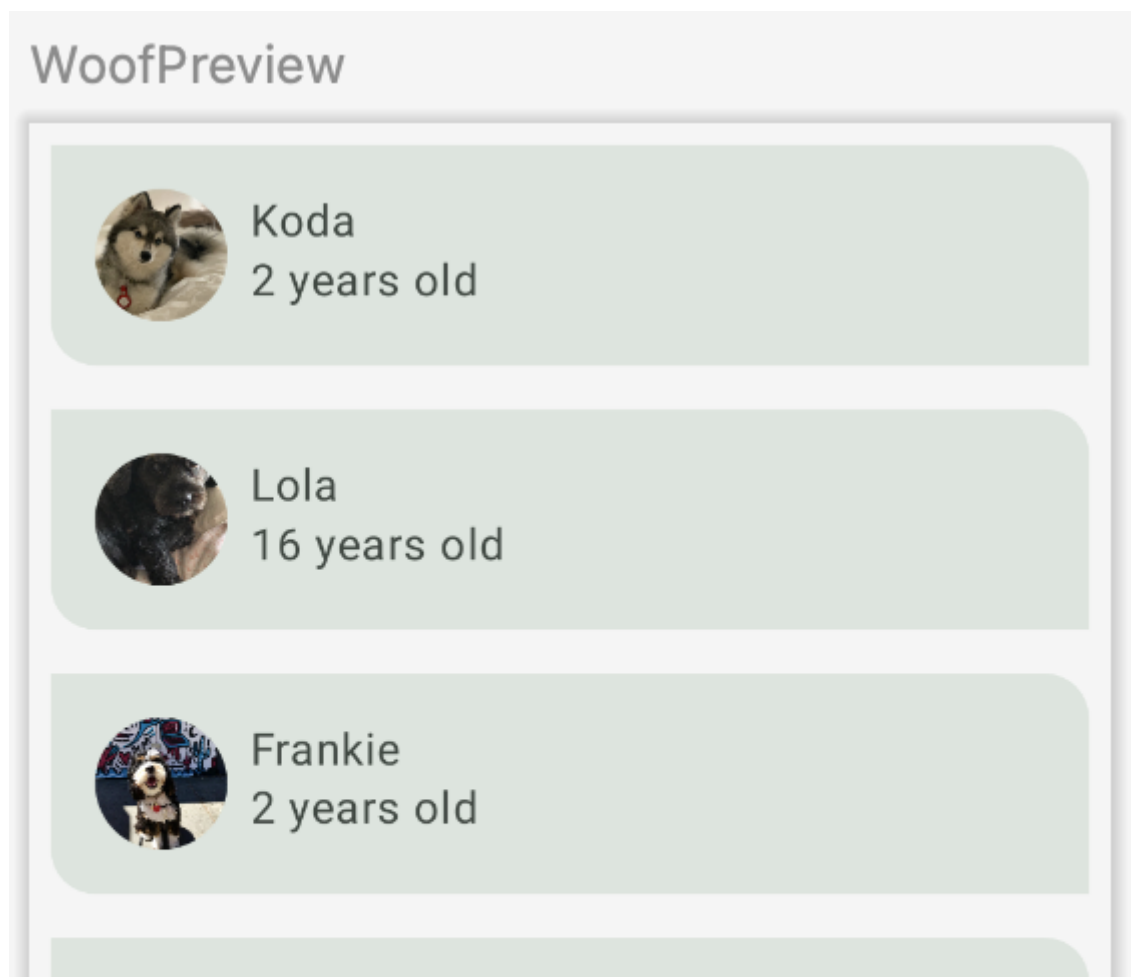
опции для украшения. Украшение можно добавить с помощью границы, формы и других элементов. В этом разделе вы будете использовать карточку для добавления формы к элементу списка.

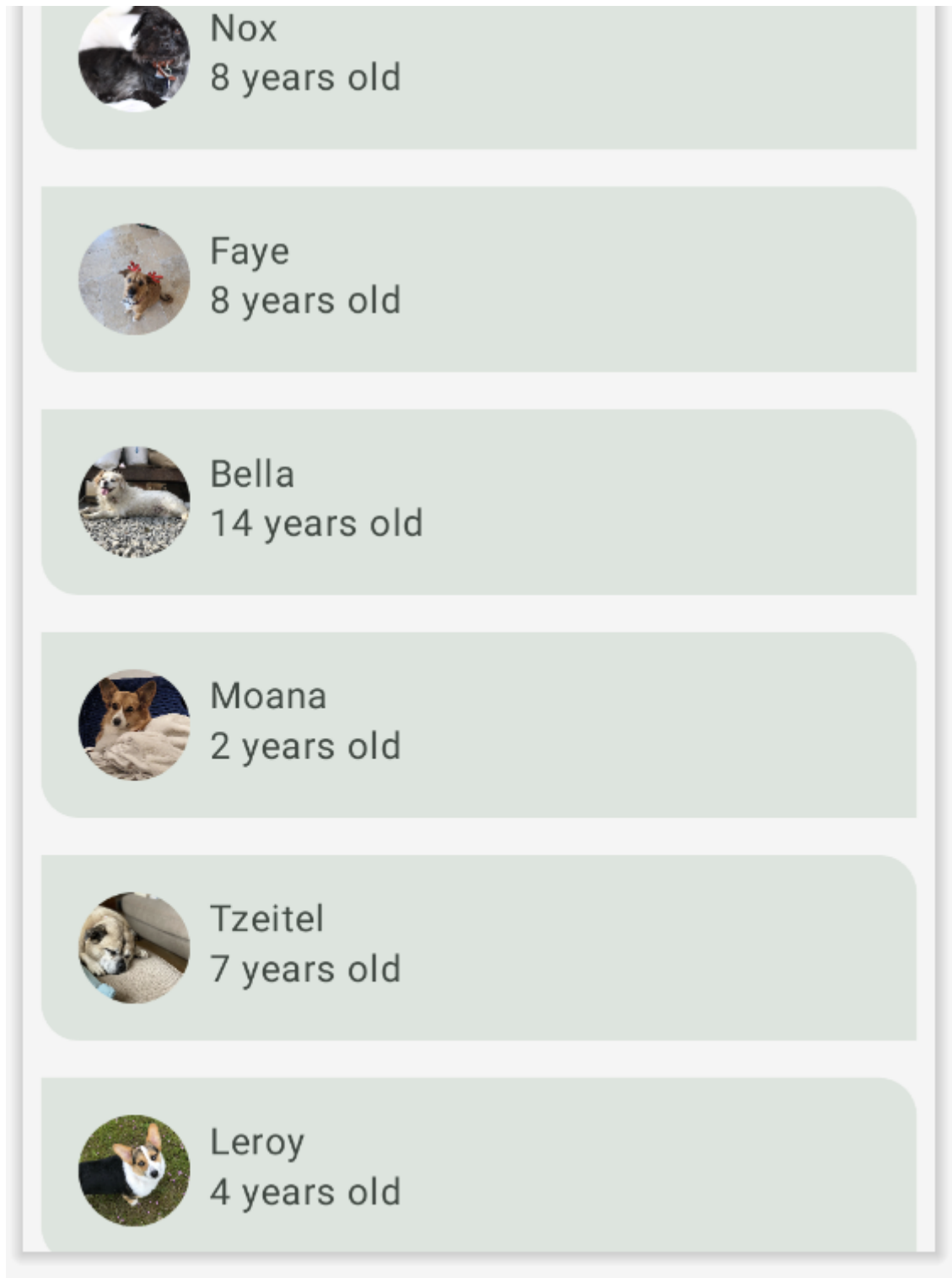


- Откройте файл `Shape.kt`. Card - это средний компонент, поэтому вы добавляете параметр `medium` объекта `Shapes`. В этом приложении верхний правый и нижний левый углы элемента списка нужно сделать полностью круглыми, но не полностью. Чтобы добиться этого, передайте в атрибут `medium` значение `16.dp`.

```
medium = RoundedCornerShape(bottomStart = 16.dp, topEnd = 16.dp)
```

Поскольку откытка по умолчанию уже использует среднюю форму, вам не нужно явно устанавливать ее на среднюю форму. Посмотрите предварительный просмотр, чтобы увидеть новую форму карточки!

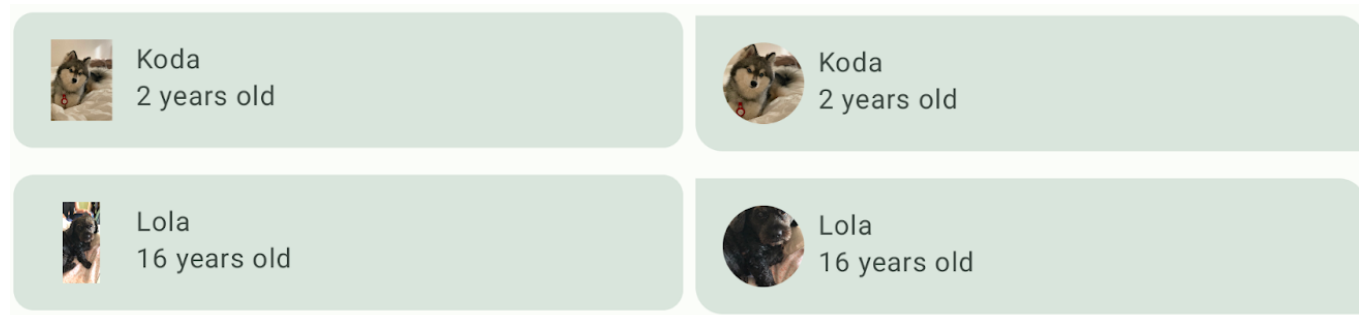




Если вы вернетесь к файлу `Theme.kt` в `WoofTheme()` и посмотрите на `MaterialTheme()`, вы увидите, что атрибут `shapes` установлен на значение `Shapes`, которое вы только что обновили.

```
MaterialTheme(  
    colors = colors,  
    typography = Typography,  
    shapes = Shapes,  
    content = content  
)
```

Ниже показаны элементы списка до и после шейпинга. Обратите внимание, насколько визуально привлекательнее стало приложение с добавлением шейпинга.



Добавьте типографику

Шкала типов Material Design

Шкала типов - это набор стилей шрифтов, которые можно использовать во всех приложениях, обеспечивая гибкий, но последовательный стиль. Шкала типов Material Design включает пятнадцать стилей шрифтов, поддерживаемых системой типов. Названия и группировки были упрощены до следующих: display, headline, title, body и label, с большим, средним и маленьким размером для каждого. Эти варианты нужно использовать только в том случае, если вы хотите настроить свое приложение. Если вы не знаете, что установить для каждой категории шкалы, знайте, что есть шкала по умолчанию, которую можно использовать.

Display Large	Roboto 57/64 . 0
Display Medium	Roboto 45/52 . 0
Display Small	Roboto 36/44 . 0
Headline Large	Roboto 32/40 . 0
Headline Medium	Roboto 28/36 . 0
Headline Small	Roboto 24/32 . 0
Title Large	NEW - Roboto Medium 22/28 . 0
Title Medium	Roboto Medium 16/24 . +0.15
Title Small	Roboto Medium 14/20 . +0.1
Body Large	Roboto 16/24 . +0.5
Body Medium	Roboto 14/20 . +0.25
Body Small	Roboto 12/16 . +0.4
Label Large	Roboto Medium 14/20 . +0.1
Label Medium	Roboto Medium 12/16 . +0.5
Label Small	NEW - Roboto Medium 11/16 . +0.5

Шкала типов содержит многократно используемые категории текста, каждая из которых имеет свое назначение и значение.

Дисплей Как самый крупный текст на экране, дисплейные стили предназначены для короткого, важного текста или цифр. Они лучше всего работают на больших экранах.

Заголовок Заголовки лучше всего подходят для короткого текста с высоким выделением на небольших экранах. Эти стили могут быть полезны для выделения основных отрывков текста или важных областей контента.

Заголовок Заголовки меньше по размеру, чем стили заголовков, и их следует использовать для текста со средним выделением, который остается относительно коротким.

Тело Стили body используются для более длинных отрывков текста в вашем приложении.

Label Стили Label - это более мелкие, утилитарные стили, которые используются для текста внутри компонентов или для очень мелкого текста в теле контента, например, для подписей.

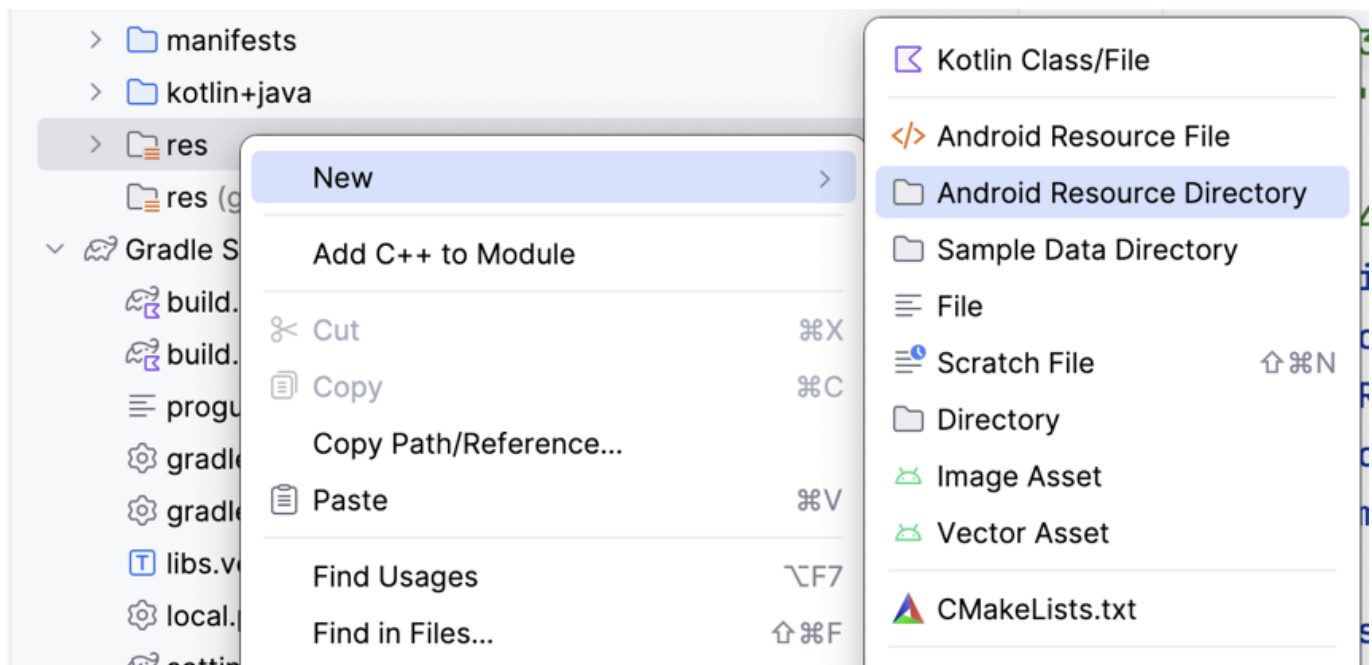
Шрифты Платформа Android предоставляет множество шрифтов, но вы можете захотеть настроить свое приложение, используя шрифт, не предусмотренный по умолчанию. Пользовательские шрифты могут придать индивидуальность и использоваться для брендинга.

В этом разделе вы добавите пользовательские шрифты **Abril Fatface**, **Montserrat Bold** и **Montserrat Regular**. Вы будете использовать заголовки `displayLarge` и `displayMedium`, а также текст `bodyLarge` из системы Material Type и добавите их в текст вашего приложения.

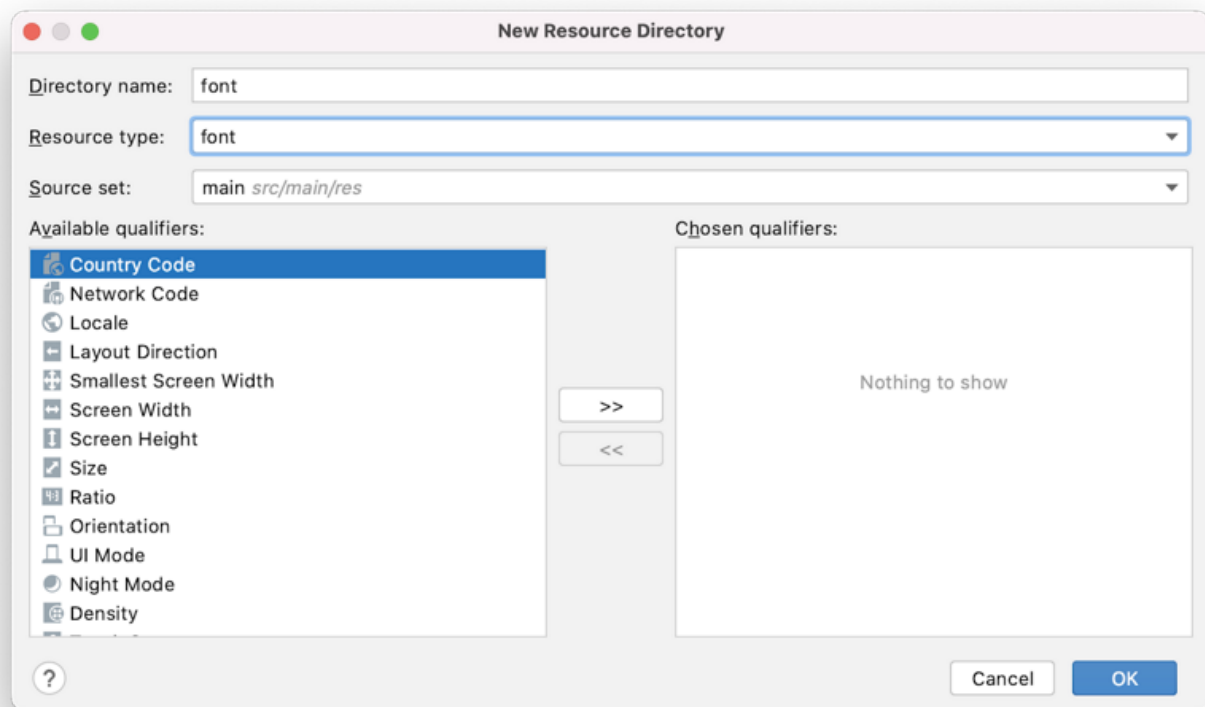
Создайте каталог ресурсов шрифта Android.

Прежде чем добавлять шрифты в приложение, необходимо добавить каталог шрифтов.

- В представлении проекта в Android Studio щелкните правой кнопкой мыши папку `res`. Выберите **New > Android Resource Directory**.



- Назовите шрифт `Directory`, установите тип ресурса как `font` и нажмите **OK**.



- Откройте каталог ресурсов нового шрифта, расположенный по адресу `res > font`.
- Загрузите пользовательские шрифты

Поскольку вы используете шрифты, не предусмотренные платформой Android, вам необходимо загрузить пользовательские шрифты.

- Перейдите на сайт <https://fonts.google.com/>.
- Найдите Montserrat и нажмите кнопку Загрузить семью. Распакуйте zip-файл.

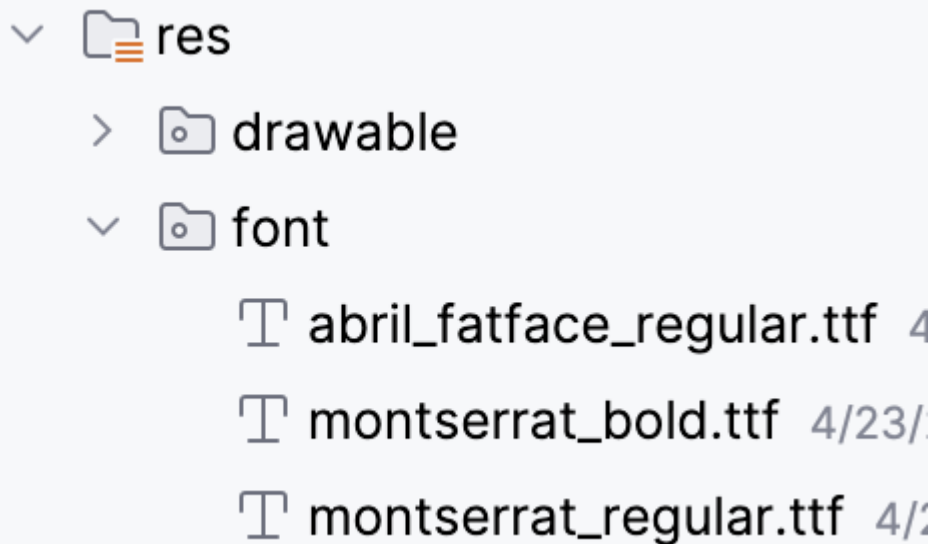
Примечание: шрифт находится в ресурсах репозитория на gogs

- Откройте скачанную папку `Montserrat`. В папке static найдите Montserrat-Bold.ttf и Montserrat-Regular.ttf (ttf означает TrueType Font и является форматом для файлов шрифтов). Выберите оба шрифта и перетащите их в каталог ресурсов шрифтов в вашем проекте в Android Studio.



- В папке со шрифтами переименуйте Montserrat-Bold.ttf в montserrat_bold.ttf и Montserrat-Regular.ttf в montserrat_regular.ttf.
- Найдите Abril Fatface и нажмите кнопку Загрузить семью.
- Откройте загруженную папку Abril_Fatface. Выберите AbrilFatface-Regular.ttf и перетащите его в каталог ресурсов шрифта.
- В папке со шрифтами переименуйте Abril_Fatface-Regular.ttf в abril_fatface_regular.ttf.

Вот как должен выглядеть каталог ресурсов шрифтов в вашем проекте с тремя пользовательскими файлами шрифтов:



Инициализация шрифтов

В окне проекта откройте `ui.theme > Type.kt`. Инициализируйте загруженные шрифты ниже операторов импорта и выше `Typography` val. Сначала инициализируйте Abril Fatface, установив его равным `FontFamily` и передав `Font` с файлом шрифта `abril_fatface_regular`.

```
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import com.example.woof.R

val AbrilFatface = FontFamily(
    Font(R.font.abril_fatface_regular)
)
```

- Инициализируйте Montserrat, под Abril Fatface, установив его равным `FontFamily` и передав `Font` с файлом шрифта `montserrat_regular`. Для `montserrat_bold` также включите `FontWeight.Bold`. Даже если вы передаете жирную версию файла шрифта, Compose не знает, что файл является жирным, поэтому вам нужно явно связать файл с `FontWeight.Bold`.

```
import androidx.compose.ui.text.font.FontWeight

val AbrilFatface = FontFamily(
    Font(R.font.abril_fatface_regular)
)

val Montserrat = FontFamily(
    Font(R.font.montserrat_regular),
    Font(R.font.montserrat_bold, FontWeight.Bold)
)
```

- Затем установите различные типы заголовков для шрифтов, которые вы только что добавили. Объект `Typography` имеет параметры для 13 различных шрифтов, о которых говорилось выше. Вы

можете задать столько параметров, сколько вам нужно. В этом приложении мы зададим `displayLarge`, `displayMedium` и `bodyLarge`. В следующей части этого приложения вы будете использовать `labelSmall`, поэтому добавьте его сюда.

Ниже приведена таблица, в которой указаны шрифт, вес и размер каждого добавляемого заголовка.

Type Role	Font	Font weight	Size
<code>displayLarge</code>	Abril Fatface	Normal	36sp
<code>displayMedium</code>	Montserrat	Bold	20sp
<code>labelSmall</code>	Montserrat	Bold	14sp
<code>bodyLarge</code>	Montserrat	Normal	14sp

- Для атрибута `displayLarge` установите значение `TextStyle` и заполните `fontFamily`, `fontWeight` и `fontSize` информацией из таблицы выше. Это означает, что весь текст, установленный в `displayLarge`, будет иметь шрифт **Abril Fatface**, нормальный вес шрифта и размер шрифта 36.sp. Повторите этот процесс для `displayMedium`, `labelSmall` и `bodyLarge`.

```
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.unit.sp

val Typography = Typography(
    displayLarge = TextStyle(
        fontFamily = AbrilFatface,
        fontWeight = FontWeight.Normal,
        fontSize = 36.sp
    ),
    displayMedium = TextStyle(
        fontFamily = Montserrat,
        fontWeight = FontWeight.Bold,
        fontSize = 20.sp
    ),
    labelSmall = TextStyle(
        fontFamily = Montserrat,
        fontWeight = FontWeight.Bold,
        fontSize = 14.sp
    ),
    bodyLarge = TextStyle(
        fontFamily = Montserrat,
        fontWeight = FontWeight.Normal,
```

```
        fontSize = 14.sp
    )
}
```

- Если вы перейдете в файл `Theme.kt` в `WoofTheme()` и посмотрите на `MaterialTheme()`, параметр `typography` будет равен параметру `Typography val`, который вы только что обновили.

```
MaterialTheme(
    colors = colors,
    typography = Typography,
    shapes = Shapes,
    content = content
)
```

- Добавьте типографику в текст приложения

Теперь вы добавите типы заголовков к каждому экземпляру текста в приложении.

- Добавьте `displayMedium` в качестве стиля для имени собаки, потому что это короткий и важный фрагмент информации. Добавьте `bodyLarge` в качестве стиля для `dogAge`, потому что он хорошо работает с меньшими размерами текста.

```
@Composable
fun DogInformation(
    @StringRes dogName: Int,
    dogAge: Int,
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier) {
        Text(
            text = stringResource(dogName),
            style = MaterialTheme.typography.displayMedium,
            modifier = Modifier.padding(top = dimensionResource(id =
R.dimen.padding_small))
        )
        Text(
            text = stringResource(R.string.years_old, dogAge),
            style = MaterialTheme.typography.bodyLarge
        )
    }
}
```

Теперь в `WoofPreview()` имя собаки отображается жирным шрифтом `Montserrat` размером `20.sp`, а возраст собаки - обычным шрифтом `Montserrat` размером `14.sp`.

WoofPreview



Koda

2 years old



Lola

16 years old



Frankie

2 years old



Nox

8 years old



Faye

8 years old



Bella

14 years old



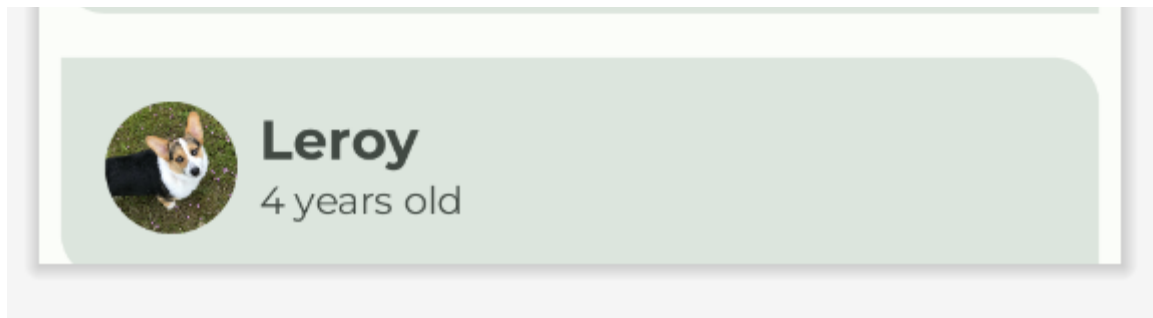
Moana

2 years old

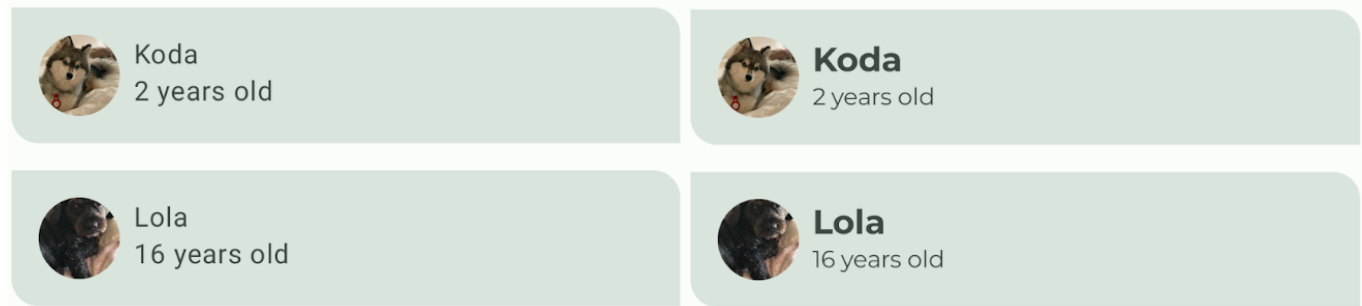


Tzeitel

7 years old



Ниже показаны элементы списка до и после добавления типографики. Обратите внимание на разницу в шрифте между кличкой собаки и ее возрастом.



Добавьте верхнюю панель

Scaffold - это макет, который предоставляет **слоты** для различных компонентов и элементов экрана, таких как изображение, строка или столбец. В макете Scaffold также есть слот для **TopAppBar**, который вы будете использовать в этом разделе.

TopAppBar может использоваться для многих целей, но в данном случае вы будете использовать его для брендинга и придания индивидуальности вашему приложению. Существует четыре различных типа **TopAppBar**: центральный, маленький, средний и большой. В этом уроке вы реализуете центральную верхнюю панель приложений. Вы создадите композит, который будет выглядеть как на скриншоте ниже, и вставите его в секцию topBar скаффолда.



Для этого приложения наша верхняя панель состоит из строки с изображением логотипа и текста названия приложения. На логотипе изображена милая градиентная лапа и название приложения!



- Добавьте изображение и текст в верхнюю панель В файле MainActivity.kt создайте компонент WoofTopAppBar() с дополнительным модификатором.

```
@Composable
fun WoofTopAppBar(modifier: Modifier = Modifier) {

}
```

Scaffold поддерживает параметр `contentWindowInsets`, с помощью которого можно указать вставки для содержимого скаффолда. **WindowInsets** - это части экрана, где ваше приложение может пересекаться с системным пользовательским интерфейсом, которые должны быть переданы в слот контента через параметры `PaddingValues`.

Значение `contentWindowInsets` передается в `LazyColumn` в качестве `contentPadding`.

```
@Composable
fun WoofApp() {
    Scaffold { it ->
        LazyColumn(contentPadding = it) {
            items(dogs) {
                DogItem(
                    dog = it,
                    modifier =
Modifier.padding(dimensionResource(R.dimen.padding_small))
                )
            }
        }
    }
}
```

Внутри `Scaffold` добавьте атрибут `topBar` и задайте ему значение `WoofTopAppBar()`.

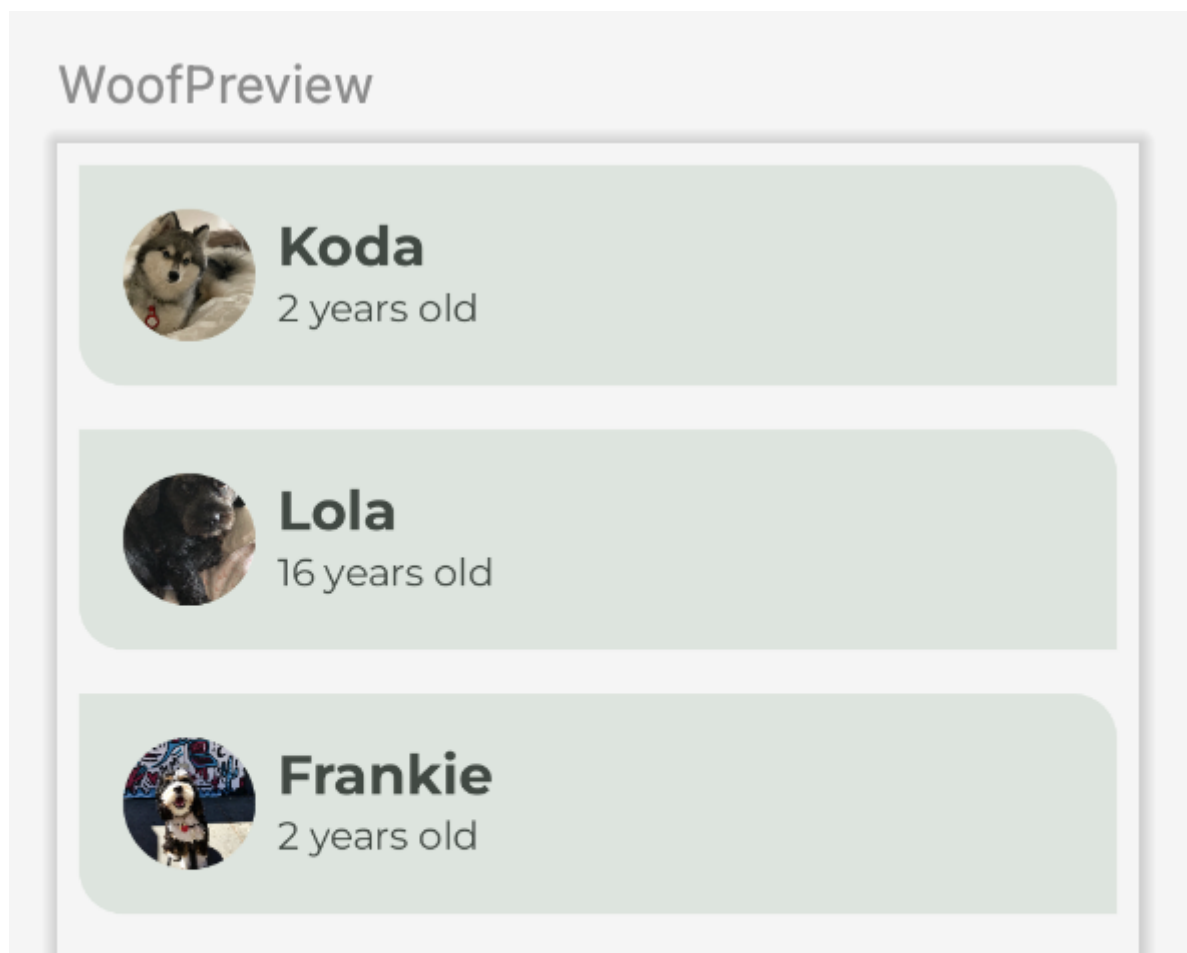
```
Scaffold(
    topBar = {
        WoofTopAppBar()
    }
)
```

```
}
)
```

Ниже показано, как будет выглядеть композит `WoofApp()`:

```
@Composable
fun WoofApp() {
    Scaffold(
        topBar = {
            WoofTopAppBar()
        }
    ) { it ->
        LazyColumn(contentPadding = it) {
            items(dogs) {
                DogItem(
                    dog = it,
                    modifier =
Modifier.padding(dimensionResource(R.dimen.padding_small))
                )
            }
        }
    }
}
```

В `WoofPreview()` ничего не изменилось, потому что ничего нет в `WoofTopAppBar()`. Давайте изменим это!



**Nox**

8 years old

**Faye**

8 years old

**Bella**

14 years old

**Moana**

2 years old

**Tzeitel**

7 years old

**Leroy**

4 years old

- В составной части `WoofTopAppBar()` добавьте `CenterAlignedTopAppBar()` и установите параметр модификатора на модификатор, переданный в `WoofTopAppBar()`.

```
import androidx.compose.material3.CenterAlignedTopAppBar
```

```
@Composable
```

```
fun WoofTopAppBar(modifier: Modifier = Modifier) {  
    CenterAlignedTopAppBar(  
        modifier = modifier
```

```
)
}
```

- Для параметра title передайте строку Row, которая будет содержать изображение и текст CenterAlignedTopAppBar.

```
@Composable
fun WoofTopAppBar(modifier: Modifier = Modifier){
    CenterAlignedTopAppBar(
        title = {
            Row() {

            }
        },
        modifier = modifier
    )
}
```

Добавьте изображение логотипа в строку.

- Установите размер изображения в модификаторе как image_size в файле dimens.xml и padding как padding_small из файла dimens.xml. С помощью painter установите изображение как ic_woof_logo из папки drawable.
- Установите contentDescription как null. В данной ситуации логотип приложения не добавляет никакой семантической информации для пользователей с ослабленным зрением, поэтому нам не нужно добавлять описание содержимого.

```
Row() {
    Image(
        modifier = Modifier
            .size(dimensionResource(id = R.dimen.image_size))
            .padding(dimensionResource(id = R.dimen.padding_small)),
        painter = painterResource(R.drawable.ic_woof_logo),
        contentDescription = null
    )
}
```

- Затем добавьте текстовый компонент в строку после изображения. С помощью функции stringResource() установите для него значение app_name. Это установит текст на название приложения, которое хранится в файле strings.xml.
- Установите стиль текста на displayLarge, поскольку название приложения - это короткий и важный текст.


```
Text(
    text = stringResource(R.string.app_name),
    style = MaterialTheme.typography.displayLarge
)
```

WoofPreview



Koda

2 years old



Lola

16 years old



Frankie

2 years old



Nox

8 years old



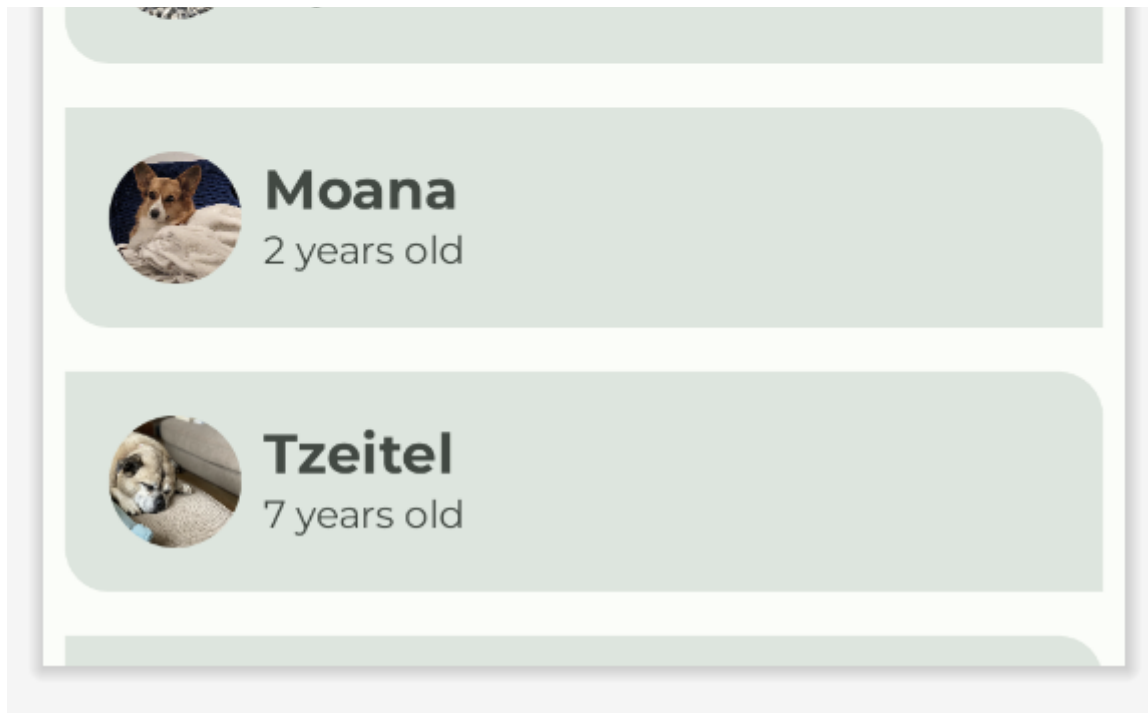
Faye

8 years old



Bella

14 years old



Вот что отображается в `WoofPreview()`, это выглядит немного не так, потому что иконка и текст не выровнены по вертикали.

- Чтобы исправить это, добавьте в `Row` значение-параметр `verticalAlignment` и установите его равным `Alignment.CenterVertically`.

```
import androidx.compose.ui.Alignment

Row(
    verticalAlignment = Alignment.CenterVertically
)
```

WoofPreview



Koda

2 years old



Lola

16 years old



Frankie

2 years old

Это выглядит намного лучше!

Это полная композиция `WoofTopAppBar()`:

```
@Composable
fun WoofTopAppBar(modifier: Modifier = Modifier) {
    CenterAlignedTopAppBar(
        title = {
            Row(
                verticalAlignment = Alignment.CenterVertically
            ) {
                Image(
                    modifier = Modifier
                        .size(dimensionResource(id = R.dimen.image_size))
                        .padding(dimensionResource(id = R.dimen.padding_small)),
                    painter = painterResource(R.drawable.ic_woof_logo),

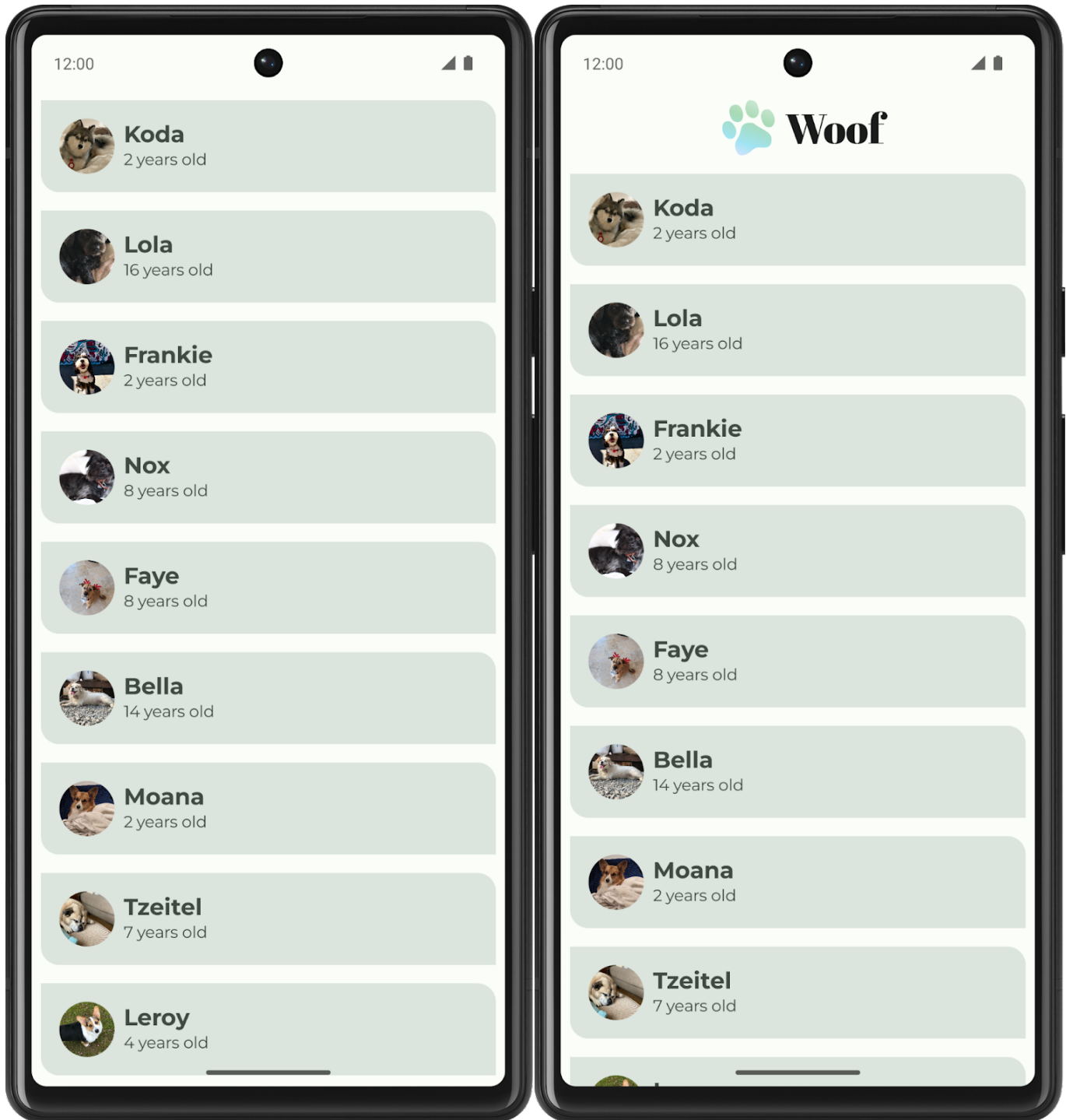
                    contentDescription = null
                )
                Text(
                    text = stringResource(R.string.app_name),
                    style = MaterialTheme.typography.displayLarge
                )
            }
        }
    )
}
```

```

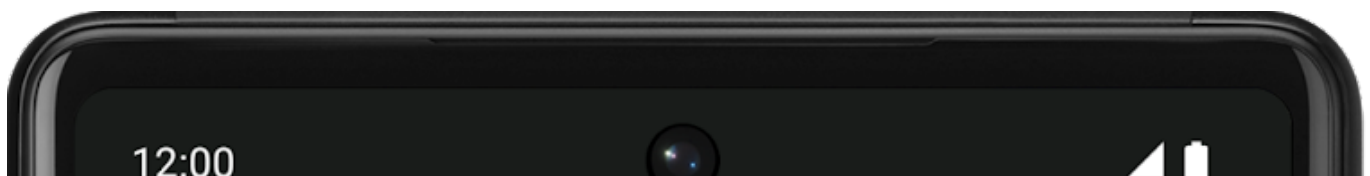
    }
  },
  modifier = modifier
)
}

```

- Запустите приложение и полюбуйтесь, как красиво TopAppBar связывает приложение воедино.



А теперь посмотрите на готовое приложение в темной теме!





Koda
2 years old



Lola
16 years old



Frankie
2 years old



Nox
8 years old



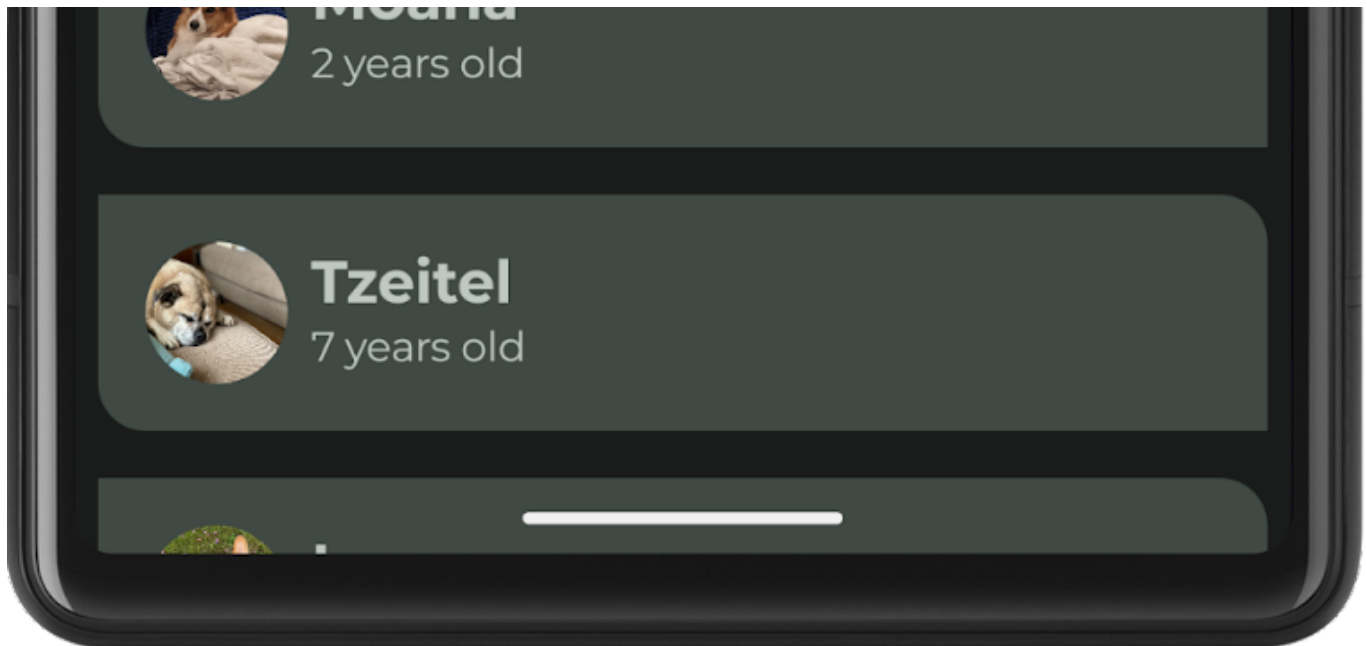
Faye
8 years old



Bella
14 years old



Moana



Заключение

Вы только что создали свое первое приложение Material! Вы создали пользовательскую цветовую палитру для светлой и темной тем, создали формы для различных компонентов, загрузили шрифты и добавили их в приложение, а также создали красивую верхнюю панель, чтобы связать все вместе. Воспользуйтесь навыками, полученными в этой работе, и измените цвета, формы и типографику, чтобы сделать приложения полностью своими!

Резюме

- **Material Theming** позволяет вам использовать Material Design в вашем приложении, с руководством по настройке цветов, типографики и форм.
- Файл **Theme.kt** - это место, где задается тема, через композитную функцию [название вашего приложения]+Theme()- WoofTheme() в случае с этим приложением. В этой функции объект MaterialTheme задает цвет, типографику, формы и содержание приложения.
- В файле Color.kt перечисляются цвета, используемые в приложении. Затем в Theme.kt вы назначаете цвета в LightColorPalette и DarkColorPalette на определенные слоты. Не все слоты должны быть назначены.
- Ваше приложение может выбрать Force Dark, что означает, что система будет реализовывать темную тему для вас. Однако для пользователей будет лучше, если вы внедрите темную тему, чтобы сохранить полный контроль над темой приложения.
- Shape.kt - это место, где вы определяете формы вашего приложения. Есть три размера форм (маленькая, средняя, большая), и вы можете указать, как будут скруглены углы.
- Формы направляют внимание, идентифицируют компоненты, передают состояние и выражают бренд.
- Type.kt - это место, где вы инициализируете свои шрифты и назначаете fontFamily, fontWeight и fontSize для шкалы типов Material Design.
- Шкала типов Material Design включает в себя ряд контрастных стилей, которые соответствуют потребностям вашего приложения и его содержимого. Шкала типов представляет собой комбинацию из 15 стилей, поддерживаемых системой типов.